63. 2D Drawing Engine (DRW)

63.1 Overview

This MCU incorporates a 2D drawing engine (DRW). Table 63.1 lists the DRW specifications.

Table 63.1 DRW Specifications

Item		Descriptions
Vector drawing engine		Extended rendering primitives supported in hardware ● Lines ● Polygons ● Circles and ellipses ● Quadratic Béziers ● Texture mapping
BitBLT function		Types of BitBLT operations ● Fill ● Copy ● Stretch BitBLT ● Rotate and scale ● Alpha blending ● Bilinear filtering ● Color conversion ● Subpixel exact placement
Color format	Frame buffer	Four types ● A (8) ● RGB (565) ● ARGB (4444) ● ARGB (8888)
	Texture	11 types ● CLUT (1)/I (1) ● CLUT (2)/I (2) ● CLUT (4)/I (4) ● CLUT (8)/I (8) ● A (8) ● ACLUT (44) ● RGB (565) ● ARGB (4444) ● ARGB (1555) ● RGB (888) ● ARGB (8888)
Texture		● Run-length encoding (RLE) unit ● 256-entry color look-up table (CLUT) ● Color keying unit
Render		Two modes for rendering process ● Register mode ● Display list mode
Data arrangement		Little endian
Performance counter		● Two independent 32-bit counters ● Trigger is selectable from 14 events
Interrupt sources		Three interrupt sources ● 2D Drawing Engine bus error interrupt (DRWBUSIRQ) ● Current render process finished interrupt (DRWENUMIRQ) ● Display list interrupt (DRWDLISTIRQ)
Module-stop function		Module-stop state can be set to reduce power consumption
TrustZone Filter		Security and Privilege attribution can be set

The 2D Drawing Engine provides two inputs (texture read and frame buffer read) and one output (frame buffer write). The internal color format is always ARGB (8888). The color formats from the inputs are converted to the internal format on read and converted back on write.

The 2D Drawing Engine also supports a display list mode, which makes it possible to decouple the CPU and graphics controller efficiently and perform rendering in parallel with other CPU activities.

Figure 63.1 shows examples of objects that can be drawn in hardware with the 2D Drawing Engine, Figure 63.2 shows a simplified rendering pipeline setup, and Figure 63.3 shows a block diagram of the module.

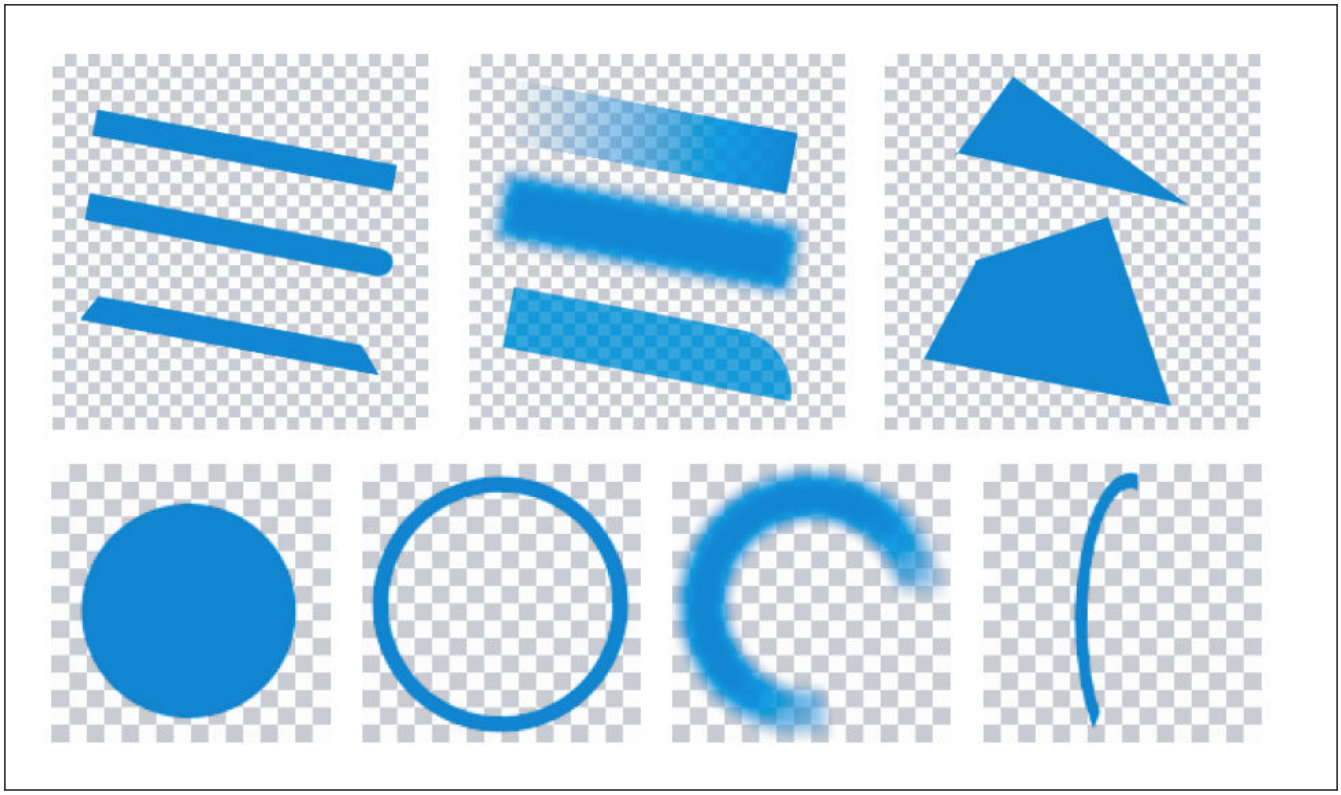


Figure 63.1 Examples of drawing objects

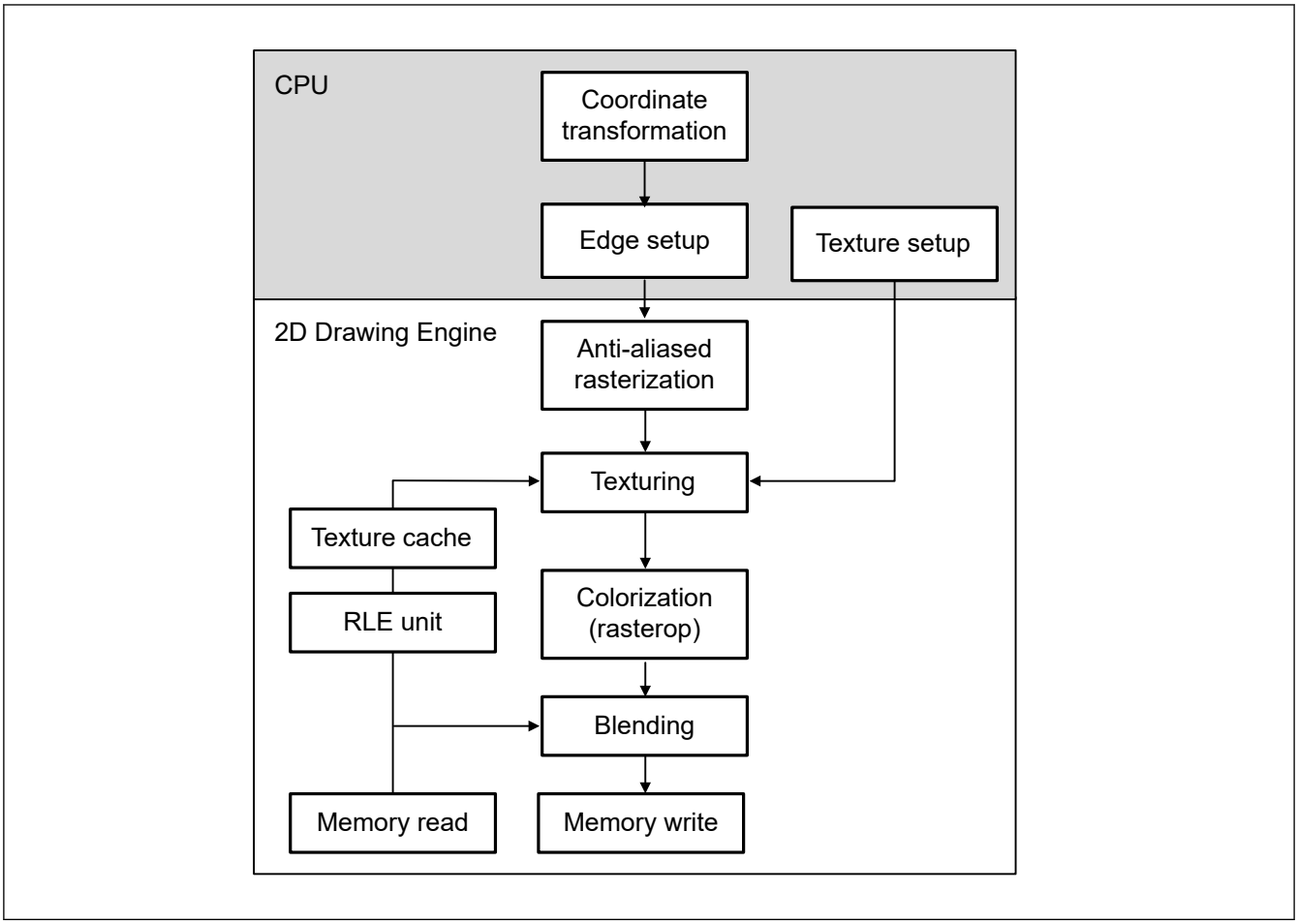


Figure 63.2 Simplified rendering pipeline setup

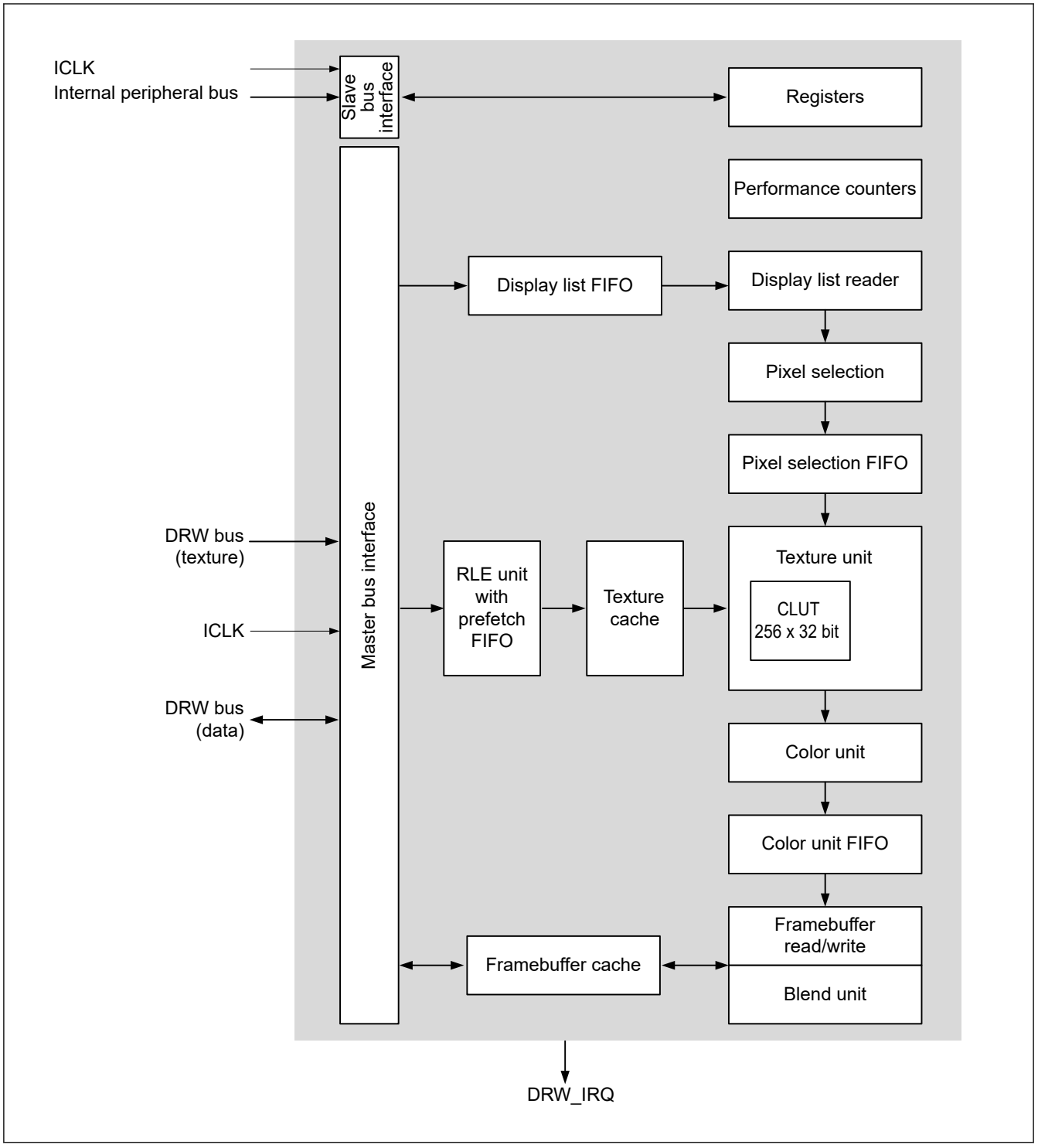


Figure 63.3 2D Drawing Engine block diagram

The 2D Drawing Engine accesses the DRW bus as a bus master through separate caches for:

- Reading and writing pixel data from and to the framebuffer
- Reading textures
- Reading display lists.

The control registers are accessed through the internal peripheral bus interface.

63.2 Register Descriptions

63.2.1 CONTROL : Geometry Control Register

Base address: DRW = 0x4044_4000
DRW_NS = 0x5044_4000

Offset address: 0x00

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	SPANSTORE	SPANABORT	UNIONCD	UNIONAB	UNIONN56	UNIONN34	UNIONN12	BAND2ENABLE
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	BAND1ENABLE	LIM6THRESHOLD	LIM5THRESHOLD	LIM4THRESHOLD	LIM3THRESHOLD	LIM2THRESHOLD	LIM1THRESHOLD	QUAD3ENABLE	QUAD2ENABLE	QUAD1ENABLE	LIM6ENABLE	LIM5ENABLE	LIM4ENABLE	LIM3ENABLE	LIM2ENABLE	LIM1ENABLE
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
0	LIM1ENABLE	Enable Limiter 1 0: Disable 1: Enable	W
1	LIM2ENABLE	Enable Limiter 2 0: Disable 1: Enable	W
2	LIM3ENABLE	Enable Limiter 3 0: Disable 1: Enable	W
3	LIM4ENABLE	Enable Limiter 4 0: Disable 1: Enable	W
4	LIM5ENABLE	Enable Limiter 5 0: Disable 1: Enable	W
5	LIM6ENABLE	Enable Limiter 6 0: Disable 1: Enable	W
6	QUAD1ENABLE	Enable Quadratic Coupling of Limiters 1 and 2 0: Disable 1: Enable	W
7	QUAD2ENABLE	Enable Quadratic Coupling of Limiters 3 and 4 0: Disable 1: Enable	W
8	QUAD3ENABLE	Enable Quadratic Coupling of Limiters 5 and 6 0: Disable 1: Enable	W
9	LIM1THRESHOLD	Enable Limiter 1 Threshold Mode 0: Disable 1: Enable	W
10	LIM2THRESHOLD	Enable Limiter 2 Threshold Mode 0: Disable 1: Enable	W
11	LIM3THRESHOLD	Enable Limiter 3 Threshold Mode 0: Disable 1: Enable	W
12	LIM4THRESHOLD	Enable Limiter 4 Threshold Mode 0: Disable 1: Enable	W

Bit	Symbol	Function	R/W
13	LIM5THRESHOLD	Enable Limiter 5 Threshold Mode 0: Disable 1: Enable	W
14	LIM6THRESHOLD	Enable Limiter 6 Threshold Mode 0: Disable 1: Enable	W
15	BAND1ENABLE	Enable Band Post Process for Limiter 1 See section 63.2.13. LmBAND : Limiter m Band Width Parameter Register (n = 1, 2). 0: Disable 1: Enable.	W
16	BAND2ENABLE	Enable Band Post Process for Limiter 2 See section 63.2.13. LmBAND : Limiter m Band Width Parameter Register (n = 1, 2). 0: Disable 1: Enable.	W
17	UNION12	Combine Limiters 1 and 2 as Union 0: Select minimum/intersect between limiters 1 and 2 1: Select maximum/union between limiters 1 and 2. The output is called A.	W
18	UNION34	Combine Limiters 3 and 4 as Union 0: Select minimum/intersect between limiters 3 and 4 1: Select maximum/union between limiters 3 and 4. The output is called B.	W
19	UNION56	Combine Limiters 5 and 6 as Union 0: Select minimum/intersect between limiters 5 and 6 1: Select maximum/union between limiters 5 and 6. The output is called D.	W
20	UNIONAB	Combine Outputs A and B as Union 0: Select minimum/intersect between limiters A and B. 1: Select maximum/union between limiters A and B. The output is called C.	W
21	UNIONCD	Combine Outputs C and D as Union 0: Select minimum/intersect between limiters C and D 1: Select maximum/union between limiters C and D. The output is final.	W
22	SPANABORT	Spanabort Shape is horizontally convex, only a single span per scan line. See (2) Spanabort. 0: Disable 1: Enable.	W
23	SPANSTORE	Spanstore 0: Disable 1: Enable. Next line span start is always equal to or left of the current line span start. See (1) Spanstore.	W
31:24	—	The write value should be 0.	W

Note: S-TYPE-3, P-TYPE-3

63.2.2 CONTROL2 : Surface Control Register

Base address: DRW = 0x4044_4000
DRW_NS = 0x5044_4000

Offset address: 0x04

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	RLEPIXELWIDTH[1:0]		BDIA	BSIA	CLUTFORMAT	COLKEYENABLE	CLUTENABLE	RLEENABLE	WRITEALPHA[1:0]		WRITEFORMAT[1:0]		READFORMAT_L[1:0]		TEXTUREFILTERY	TEXTUREFILTERX
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	TEXTURECLAMPY	TEXTURECLAMPX	BC2	BDI	BSI	BDF	BSF	WRITEFORMAT2	BDFA	BSFA	READFORMAT_H[3:2]		USEACB	PATTERNSOURCEL5	TEXTUREENABLE	PATTERNENABLE
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
0	PATTERNENABLE	Pattern Color Enable for Pixel Source Pixel source is a pattern color (blend of COLOR1 and COLOR2 depending on PATTERN and pattern index). When patterns are used to fill a primitive index into the pattern, bit mask is generated for each pixel with the U limiter. Depending on the pattern bits, the color is selected from COLOR1 and COLOR2 registers. Fractional indices can be interpolated between those two values using TEXTUREFILTERX = 1. The pattern can be wrapped using TEXTURECLAMPX = 0, and the mask must be set in the TEXMASK register using the mask for U limiter. 0: Disable pattern 1: Enable pattern.	W
1	TEXTUREENABLE	Texture Enable for Pixel Source Pixel source is read from texture and used as an alpha to blend between COLOR1 and COLOR2. 0: Disable texture 1: Enable texture.	W
2	PATTERNSOURCEL5	Limiter 5 Enable for Pattern Index Limiter 5 is used as a pattern index instead of the default U limiter. Limiter 5 can be combined with limiter 6 to form a quadratic limiter that can be used to make quadratic pattern functions to draw radial patterns.	W
3	USEACB	Alpha Blend Mode 0: Use WRITEALPHA[1:0] mode 1: Use full alpha channel blending mode.	W
5:4	READFORMAT_H[3:2]	Texture Format Descriptor Bits [3] and [2] of the texture buffer format. See the detailed description of the READFORMAT[1:0] bit in this section.	W
6	BSFA	Blend Source Factor for Alpha Channel Valid in alpha channel blending mode (USEACB = 1). 0: Use 1.0 as blend source factor for alpha channel 1: Use alpha as blend source factor for alpha channel.	W
7	BDFA	Blend Destination Factor for Alpha Channel Valid in alpha channel blending mode (USEACB = 1). 0: Use 1.0 as blend destination factor for alpha channel 1: Use alpha as blend destination factor for alpha channel.	W
8	WRITEFORMAT2	Writeback Framebuffer Format Bit [2] of framebuffer pixel format. See the description of WRITEFORMAT[1:0] in this section.	W
9	BSF	Blend Source Factor Source factor is alpha (factor is 1 per default). 0: Use 1.0 as blend source factor 1: Use alpha as blend source factor.	W
10	BDF	Blend Destination Factor Destination factor is alpha (factor is 1 per default). 0: Use 1.0 as blend destination factor 1: Use alpha as blend destination factor.	W
11	BSI	Blend Source Factor Inverted Source factor is inverted (meaning 1-a or 1-1 depending on BSF). 0: Use blend factor as specified through BSF 1: Invert blend source factor (1-x).	W
12	BDI	Blend Destination Factor Inverted Destination factor is inverted (meaning 1-a or 1-1 depending on BDF). 0: Use blend factor as specified through BDF 1: Invert blend destination factor (1-x).	W
13	BC2	Blend color 2 Select of blend color 2 instead of framebuffer pixel. 0: Use pixel from framebuffer as destination (DST) 1: Use color 2 as destination (DST).	W

<table>
<tr><th>Bit</th><th>Symbol</th><th>Function</th><th>R/W</th></tr>
<tr><td>14</td><td>TEXTURECLAMPX</td><td>Calculating U Limiter Outside Used Texture
This bit describes what happens when the U limiter (x direction in texture space) calculates a u value outside of the used texture.
0: Texture wrap mode: Integer part of the calculated value from the U limiter is AND gated with TEXUMASK, resulting in a repetition of texture in the x/u direction
1: Texture clamp mode: Texture color at the border of the texture is taken, resulting in a repetition of texture border color in the x/u direction.</td><td>W</td></tr>
<tr><td>15</td><td>TEXTURECLAMPY</td><td>Calculating V Limiter Outside Used Texture
This bit describes what happens when the V limiter (y direction in texture space) calculates a v value outside of the used texture.
0: Texture wrap mode: Integer part of the calculated value from the V limiter is AND gated with TEXVMASK, resulting in a repetition of texture in the y/v direction.
1: Texture clamp mode: Texture color at the border of the texture is taken, resulting in a repetition of texture border color in the y/v direction.</td><td>W</td></tr>
<tr><td>16</td><td>TEXTUREFILTERX</td><td>Linear Filtering on Texture U Axis
0: No filtering on texture U axis
1: Linear filtering on texture U axis.</td><td>W</td></tr>
<tr><td>17</td><td>TEXTUREFILTERY</td><td>Linear Filtering on Texture V Axis
0: No filtering on texture V axis
1: Linear filtering on texture V axis.</td><td>W</td></tr>
<tr><td>19:18</td><td>READFORMAT_L [1:0]</td><td>Texture Format Descriptor
Pixel format of the texture buffer.
<table>
<tr><td>b5</td><td>b4</td><td>b19</td><td>b18</td><td></td></tr>
<tr><td colspan="2">READFORMAT_H [3:2]]</td><td colspan="2">READFORMAT_L [1:0]</td><td></td></tr>
<tr><td>0</td><td>0</td><td>0</td><td>0</td><td>8 bpp A (8)</td></tr>
<tr><td>0</td><td>0</td><td>0</td><td>1</td><td>16 bpp RGB (565)</td></tr>
<tr><td>0</td><td>0</td><td>1</td><td>0</td><td>32 bpp ARGB (8888)</td></tr>
<tr><td>0</td><td>0</td><td>1</td><td>1</td><td>16 bpp ARGB (4444)</td></tr>
<tr><td>0</td><td>1</td><td>0</td><td>0</td><td>16 bpp ARGB (1555)</td></tr>
<tr><td>0</td><td>1</td><td>0</td><td>1</td><td>8 bpp ACLUT (44), 4 bit alpha and 4 bit indexed color</td></tr>
<tr><td>1</td><td>0</td><td>0</td><td>1</td><td>8 bpp CLUT (8)/I (8), 8 bit indexed color/ luminance</td></tr>
<tr><td>1</td><td>0</td><td>1</td><td>0</td><td>4 bpp CLUT (4)/I (4), 4 bit indexed color/ luminance</td></tr>
<tr><td>1</td><td>0</td><td>1</td><td>1</td><td>2 bpp CLUT (2)/I (2), 2 bit indexed color/ luminance</td></tr>
<tr><td>1</td><td>1</td><td>0</td><td>0</td><td>1 bpp CLUT (1)/I (1), 1 bit indexed color/ luminance.</td></tr>
<tr><td colspan="4">Others:</td><td>Setting prohibited</td></tr>
</table></td><td>W</td></tr>
<tr><td>21:20</td><td>WRITEFORMAT[1:0]</td><td>Writeback Framebuffer Format
Pixel format of the framebuffer.
<table>
<tr><td>b8</td><td>b20</td><td>b21</td><td></td></tr>
<tr><td>WRITEFORMAT2</td><td colspan="2">WRITEFORMAT[1:0]</td><td></td></tr>
<tr><td>0</td><td>0</td><td>0</td><td>8 bpp A (8)</td></tr>
<tr><td>0</td><td>0</td><td>1</td><td>16 bpp RGB (565)</td></tr>
<tr><td>0</td><td>1</td><td>0</td><td>32 bpp ARGB (8888)</td></tr>
<tr><td>0</td><td>1</td><td>1</td><td>16 bpp ARGB (4444)</td></tr>
<tr><td colspan="3">Others:</td><td>Setting prohibited</td></tr>
</table></td><td>W</td></tr>
</table>

Bit	Symbol	Function	R/W
23:22	WRITEALPHA[1:0]	Writeback Alpha Source for Framebuffer 0 0: (USEACB = 0) Use alpha from color 2 (USEACB = 0) (USEACB = 1) BC2A = 1: Use alpha in color 2 as destination (DST_A) 0 1: (USEACB = 0) Use source alpha (pixel coverage) (USEACB = 1) BC2A = 0: Use alpha from framebuffer as destination (DST_A) 1 0: (USEACB = 0) Use 0.0 as alpha (USEACB = 1) BC2A = 0: Use alpha from framebuffer as destination (DST_A) 1 0: (USEACB = 0) Use alpha from framebuffer (USEACB = 1) BC2A = 0: Use alpha from framebuffer as destination (DST_A)	W
24	RLEENABLE	RLE Enable 0: Disable RLE 1: Enable RLE.	W
25	CLUTENABLE	CLUT Enable If CLUTENABLE = 0 (CLUT disabled), the index is directly put on the RGB channels. 0: Disable CLUT 1: Enable CLUT	W
26	COLKEYENABLE	Color Keying Enable 0: Disable color keying 1: Enable color keying.	W
27	CLUTFORMAT	CLUT Format 0: Format CLUT as ARGB (8888) 1: Format CLUT as RGB (565).	W
28	BSIA	Blend Source Factor Inverted in Alpha Channel In alpha channel blending mode (USEACB = 1): 0: Use blend factor as specified through BSFA 1: Invert blend source factor (1-x).	W
29	BDIA	Blend Destination Factor Inverted in Alpha Channel In alpha channel blending mode (USEACB = 1): 0: Use blend factor as specified through BDFA 1: Invert destination blend factor (1-x).	W
31:30	RLEPIXELWIDTH [1:0]	Texel Width for RLE Unit 0 0: 1 byte per texel 0 1: 2 bytes per texel 1 0: 3 bytes per texel 1 1: 4 bytes per texel	W

Note: S-TYPE-3, P-TYPE-3

63.2.3 IRQCTL : Interrupt Control Register

Base address: DRW = 0x4044_4000
DRW_NS = 0x5044_4000

Offset address: 0xC0

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	—	—	BUSIRQCLR	BUSIRQEN	DLISTIRQCLR	ENUMIRQCLR	DLISTIRQEN	ENUMIRQEN
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
0	ENUMIRQEN	ENUMIRQ Interrupt Mask Enable 0: Disable (mask) ENUMIRQ enumeration interrupt 1: Enable (unmask) ENUMIRQ enumeration interrupt.	W

Bit	Symbol	Function	R/W
1	DLISTIRQEN	DLISTIRQ Interrupt Mask Enable 0: Disable (mask) DLISTIRQ display list interrupt 1: Enable (unmask) DLISTIRQ display list interrupt.	W
2	ENUMIRQCLR	Clear ENUMIRQ 0: Do not clear ENUMIRQ enumeration interrupt 1: Clear ENUMIRQ enumeration interrupt.	W
3	DLISTIRQCLR	Clear DLISTIRQ 0: Do not clear DLISTIRQ display list interrupt 1: Clear DLISTIRQ display list interrupt.	W
4	BUSIRQEN	BUSIRQ Interrupt Mask Enable 0: Disable (mask) BUSIRQ bus error interrupt 1: Enable (unmask) BUSIRQ bus error interrupt.	W
5	BUSIRQCLR	Clear BUSIRQ 0: Do not clear BUSIRQ bus error interrupt 1: Clear BUSIRQ bus error interrupt.	W
31:6	—	The write value should be 0.	W

Note: S-TYPE-3, P-TYPE-3

63.2.4 CACHECTL : Cache Control Register

Base address: DRW = 0x4044_4000
DRW_NS = 0x5044_4000

Offset address: 0xC4

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	CFLUSHTX	CENABLETX	CFLUSHFX	CENABLEFX
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
0	CENABLEFX	Framebuffer Cache Enable 0: Disable the framebuffer cache 1: Enable the framebuffer cache.	W
1	CFLUSHFX	Flush Framebuffer Cache 0: Do not flush the framebuffer cache 1: Flush the framebuffer cache.	W
2	CENABLETX	Texture Cache Enable 0: Disable the texture cache 1: Enable the texture cache.	W
3	CFLUSHTX	Flush Texture Cache 0: Do not flush the texture cache 1: Flush the texture cache.	W
31:4	—	The write value should be 0.	W

Note: S-TYPE-3, P-TYPE-3

63.2.5 STATUS : Status Control Register

Base address: DRW = 0x4044_4000
DRW_NS = 0x5044_4000

Offset address: 0x00

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	BUSERRMDL	BUSERRMTXMRL	BUSERRMFB	—	BUSIRQ	DLISTIRQ	ENUMIRQ	DLISTACTIVE	CACHEDIRTY	BUSYWRITE	BUSYENUM
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
0	BUSYENUM	Enumeration Unit Status 0: Enumeration unit idle 1: Enumeration unit is busy, new primitive cannot be started.	R
1	BUSYWRITE	Framebuffer Writeback Status 0: Framebuffer writeback finished 1: Framebuffer writeback busy, framebuffer type cannot be changed.	R
2	CACHEDIRTY	Framebuffer Cache Status 0: Framebuffer cache is not dirty 1: Framebuffer cache is dirty, and frame should not be flipped.	R
3	DLISTACTIVE	Display List Reader Status 0: Display list reader is idle 1: Display list reader is busy, and no direct write access to registers allowed.	R
4	ENUMIRQ	Enumeration Interrupt Triggered 0: Enumeration not finished or interrupt disabled 1: Enumeration finished interrupt triggered.	R
5	DLISTIRQ	Display List Interrupt Triggered 0: Display list not finished or interrupt disabled 1: Display list finished interrupt triggered.	R
6	BUSIRQ	Bus Error Interrupt Triggered 0: No bus error occurred or interrupt disabled 1: Bus error interrupt triggered.	R
7	—	This bit is read as 0.	R
8	BUSERRMFB	Framebuffer Bus Error Interrupt Triggered 0: No framebuffer bus error occurred or interrupt disabled 1: Framebuffer bus error interrupt triggered.	R
9	BUSERRMTXMRL	Texture Bus Error Interrupt Triggered*1 0: No texture bus error occurred or interrupt disabled 1: Texture bus error interrupt triggered.	R
10	BUSERRMDL	Display List Bus Error Interrupt Triggered 0: No display list bus error occurred or interrupt disabled 1: Display list bus error interrupt triggered.	R
31:11	—	These bits are read as 0.	R

Note: S-TYPE-3, P-TYPE-3
Note 1. Because the RLE unit is also reading data through the texture bus, an error during RLE data access is also reflected in this bit.

63.2.6 HWREVISION : Hardware Version and Feature Set ID Register

Base address: DRW = 0x4044_4000
DRW_NS = 0x5044_4000

Offset address: 0x04

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	ACBLEND	—	COLORKEY	TEXCLUT256	RLEUNIT	—	TEXCLUT	PERFCOUNT	TXCACHE	FBCACHE	DLR	—
Value after reset:	0	0	0	0	1	1	1	1	1	0	1	1	1	1	1	0

Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	REV[11:0]											
Value after reset:	0	0	0	0	0	0	0	1	0	0	0	0	0	1	1	1

Bit	Symbol	Function	R/W
11:0	REV[11:0]	Revision Number of DRW is stored.	R
16:12	—	These bits are read as 0.	R
17	DLR	Display List Reader Available 0: No display list reader 1: Display list reader is available	R
18	FBCACHE	Framebuffer Cache Available 0: No framebuffer cache 1: Framebuffer cache is available	R
19	TXCACHE	Texture Cache Available 0: No texture cache 1: Texture cache is available	R
20	PERFCOUNT	Two Performance Counter Available 0: No performance counter 1: Tow performance counters available	R
21	TEXCLUT	Texture CLUT Available 0: No texture CLUT 1: Texture CLUT is available	R
22	—	This bit is read as 0.	R
23	RLEUNIT	RLE Unit Available 0: No RLE unit 1: RLE unit is available	R
24	TEXCLUT256	Texture CLUT size 0: Texture CLUT size is 16 entries 1: Texture CLUT size is 256 entries	R
25	COLORKEY	Color Key Available 0: No color key 1: Color key is available	R
26	—	This bit is read as 1.	R
27	ACBLEND	Alpha Channel Blending Available 0: Full alpha channel blending is not available 1: Full alpha channel blending is available	R
31:28	—	These bits are read as 0.	R

Note: S-TYPE-3, P-TYPE-3

63.2.7 COLOR1 : Base Color Register

Base address: DRW = 0x4044_4000
DRW_NS = 0x5044_4000

Offset address: 0x64

Bit position:	31							24	23							16	15							8	7							0
Bit field:	COLOR1A[7:0]								COLOR1R[7:0]								COLOR1G[7:0]								COLOR1B[7:0]							
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
7:0	COLOR1B[7:0]	Blue Channel of Color 1 Specifies blue channel of color 1.	W
15:8	COLOR1G[7:0]	Green Channel of Color 1	W
23:16	COLOR1R[7:0]	Red Channel of Color 1	W
31:24	COLOR1A[7:0]	Alpha Channel of Color 1 Specifies alpha channel of color 1. 0x00: Transparent ⋮ 0xFF: Opaque.	W

Note: S-TYPE-3, P-TYPE-3

63.2.8 COLOR2 : Secondary Color Register

Base address: DRW = 0x4044_4000
DRW_NS = 0x5044_4000

Offset address: 0x68

Bit position:	31							24	23							16	15							8	7							0
Bit field:	COLOR2A[7:0]								COLOR2R[7:0]								COLOR2G[7:0]								COLOR2B[7:0]							
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

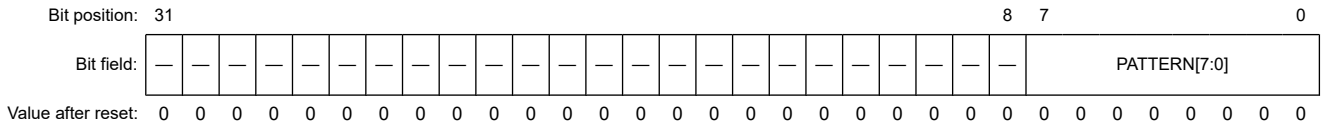
Bit	Symbol	Function	R/W
7:0	COLOR2B[7:0]	Blue Channel of Color 2 Specifies blue channel of color 2.	W
15:8	COLOR2G[7:0]	Green Channel of Color 2 Specifies green channel of color 2.	W
23:16	COLOR2R[7:0]	Red Channel of Color 2 Specifies red channel of color 2.	W
31:24	COLOR2A[7:0]	Alpha Channel of Color 2 Specifies alpha channel of color 2. 0x00: Transparent ⋮ 0xFF: Opaque.	W

Note: S-TYPE-3, P-TYPE-3

63.2.9 PATTERN : Pattern Register

Base address: DRW = 0x4044_4000
DRW_NS = 0x5044_4000

Offset address: 0x74



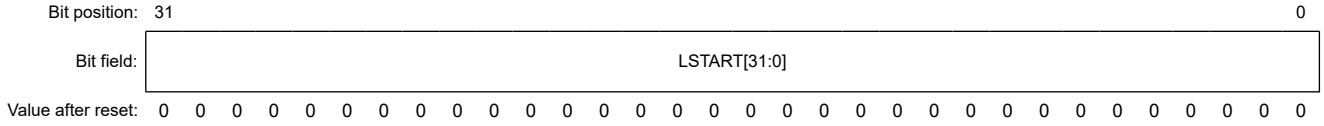
Bit	Symbol	Function	R/W
7:0	PATTERN[7:0]	Bitmap of the Pattern Specifies bitmap of the pattern.	W
31:8	—	The write value should be 0.	W

Note: S-TYPE-3, P-TYPE-3

63.2.10 LnSTART : Limiter n Start Value Register (n = 1 to 6)

Base address: DRW = 0x4044_4000
DRW_NS = 0x5044_4000

Offset address: 0x10 + 0x4 × (n - 1)



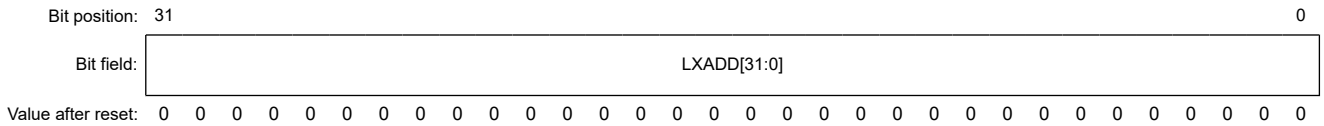
Bit	Symbol	Function	R/W
31:0	LSTART[31:0]	Start Value of the nth Limiter Specifies start value of the nth limiter.	W

Note: S-TYPE-3, P-TYPE-3

63.2.11 LnXADD : Limiter n X-Axis Increment Register (n = 1 to 6)

Base address: DRW = 0x4044_4000
DRW_NS = 0x5044_4000

Offset address: 0x28 + 0x4 × (n - 1)



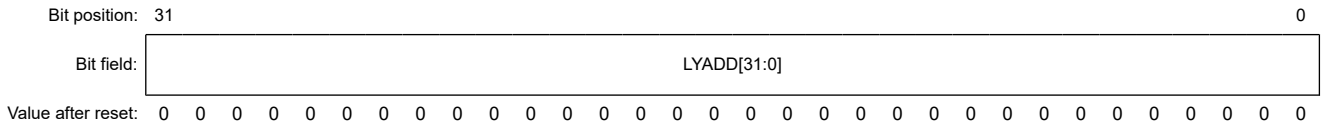
Bit	Symbol	Function	R/W
31:0	LXADD[31:0]	X-Axis Increment Specifies x-axis increment.	W

Note: S-TYPE-3, P-TYPE-3

63.2.12 LnYADD : Limiter n Y-Axis Increment Register (n = 1 to 6)

Base address: DRW = 0x4044_4000
DRW_NS = 0x5044_4000

Offset address: 0x40 + 0x4 × (n - 1)



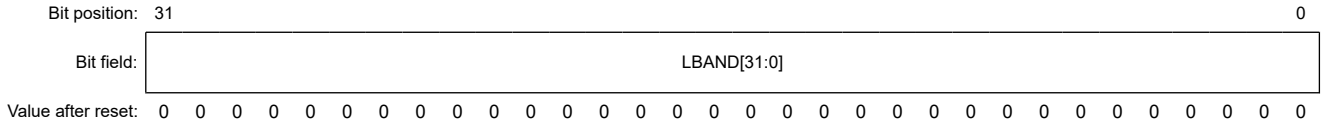
Bit	Symbol	Function	R/W
31:0	LYADD[31:0]	Y-Axis Increment Specifies y-axis increment.	W

Note: S-TYPE-3, P-TYPE-3

63.2.13 LmBAND : Limiter m Band Width Parameter Register (n = 1, 2)

Base address: DRW = 0x4044_4000
DRW_NS = 0x5044_4000

Offset address: 0x58 + 0x4



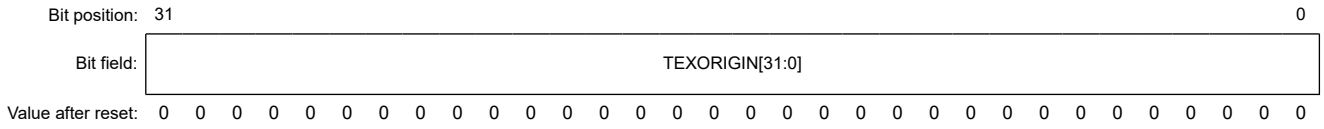
Bit	Symbol	Function	R/W
31:0	LBAND[31:0]	Limiter m Band Width Parameter Specifies limiter m band width parameter.	W

Note: S-TYPE-3, P-TYPE-3

63.2.14 TEXORIGIN : Texture Base Address Register

Base address: DRW = 0x4044_4000
DRW_NS = 0x5044_4000

Offset address: 0xBC



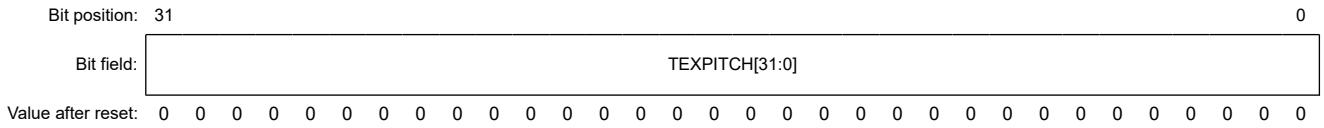
Bit	Symbol	Function	R/W
31:0	TEXORIGIN[31:0]	Texture Base Address Specifies texture base address.	W

Note: S-TYPE-3, P-TYPE-3

63.2.15 TEXPITCH : Texels Per Texture Line Register

Base address: DRW = 0x4044_4000
DRW_NS = 0x5044_4000

Offset address: 0xB4



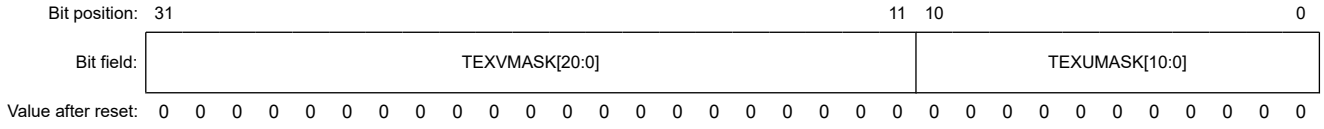
Bit	Symbol	Function	R/W
31:0	TEXPITCH[31:0]	Texels Per Texture Line Specifies texels per texture line. Valid range is from 0 to 2048.	W

Note: S-TYPE-3, P-TYPE-3

63.2.16 TEXMASK : Texture Size or Texture Address Mask Register

Base address: DRW = 0x4044_4000
DRW_NS = 0x5044_4000

Offset address: 0xB8



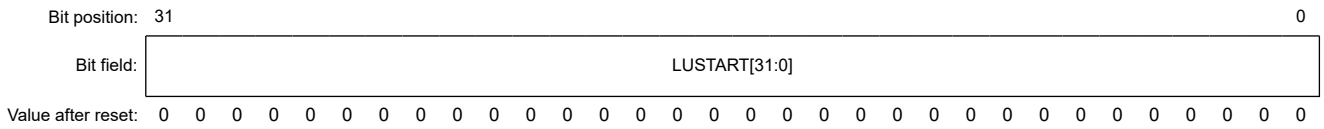
Bit	Symbol	Function	R/W
10:0	TEXUMASK[10:0]	U Mask in Texture Mode Specifies the U mask. Set to texture_width -1. ● In texture wrapping mode (CONTROL2.TEXTURECLAMPX = 0): Texture_width must be a power of 2 ● In texture clamping mode (CONTROL2.TEXTURECLAMPX = 1): All widths up to 2048 are allowed.	W
31:11	TEXVMASK[20:0]	V Mask in Texture Mode Specifies the V mask. Set to DRWTEXPITCH (texture_height - 1). ● In texture wrapping mode (CONTROL2.TEXTURECLAMPY = 0): Texture_height must be a power of 2 ● In texture clamping mode (CONTROL2.TEXTURECLAMPY = 1): All heights up to 1024 are allowed.	W

Note: S-TYPE-3, P-TYPE-3

63.2.17 LUSTART : U Limiter Start Value Register

Base address: DRW = 0x4044_4000
DRW_NS = 0x5044_4000

Offset address: 0x90



Bit	Symbol	Function	R/W
31:0	LUSTART[31:0]	U Limiter Start Value Specifies U limiter start value.	W

Note: S-TYPE-3, P-TYPE-3

63.2.18 LUXADD : U Limiter X-Axis Increment Register

Base address: DRW = 0x4044_4000
DRW_NS = 0x5044_4000

Offset address: 0x94

Bit position:	31																												0
Bit field:	LUXADD[31:0]																												
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
31:0	LUXADD[31:0]	U Limiter X-Axis Increment Specifies U limiter x-axis increment.	W

Note: S-TYPE-3, P-TYPE-3

63.2.19 LUYADD : U Limiter Y-Axis Increment Register

Base address: DRW = 0x4044_4000
DRW_NS = 0x5044_4000

Offset address: 0x98

Bit position:	31																												0
Bit field:	LUYADD[31:0]																												
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
31:0	LUYADD[31:0]	U Limiter Y-Axis Increment Specifies U limiter y-axis increment.	W

Note: S-TYPE-3, P-TYPE-3

63.2.20 LVSTARTI : V Limiter Start Value Integer Part Register

Base address: DRW = 0x4044_4000
DRW_NS = 0x5044_4000

Offset address: 0x9C

Bit position:	31																												0
Bit field:	LVSTARTI[31:0]																												
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

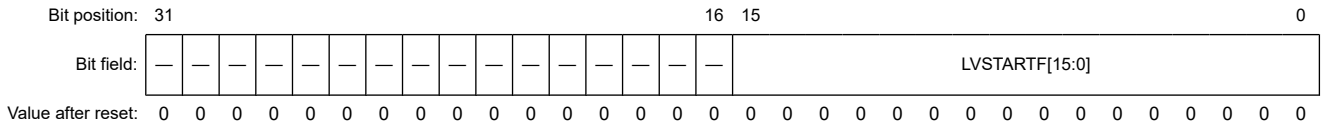
Bit	Symbol	Function	R/W
0	LVSTARTI[31:0]	V Limiter Start Value Integer Part Specifies integer part of V limiter start value.	W

Note: S-TYPE-3, P-TYPE-3

63.2.21 LVSTARTF : V Limiter Start Value Fractional Part Register

Base address: DRW = 0x4044_4000
DRW_NS = 0x5044_4000

Offset address: 0xA0



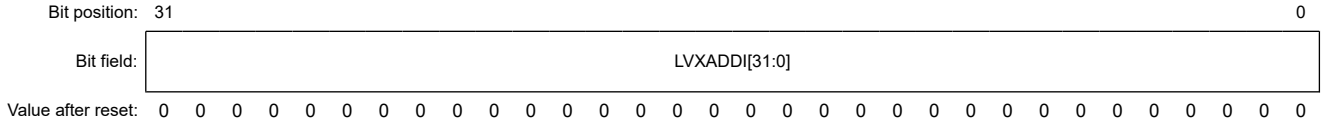
Bit	Symbol	Function	R/W
15:0	LVSTARTF[15:0]	V Limiter Start Value Fractional Part Specifies fractional part of V limiter start value.	W
31:16	—	The write value should be 0.	W

Note: S-TYPE-3, P-TYPE-3

63.2.22 LVXADDI : V Limiter X-Axis Increment Integer Part Register

Base address: DRW = 0x4044_4000
DRW_NS = 0x5044_4000

Offset address: 0xA4



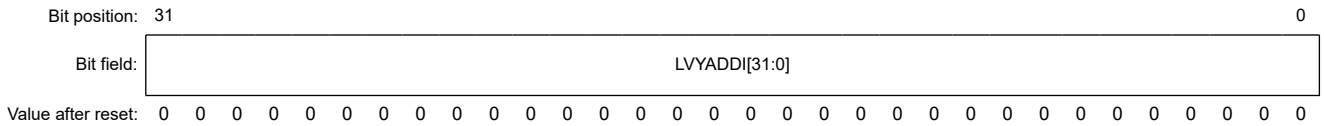
Bit	Symbol	Function	R/W
31:0	LVXADDI[31:0]	V Limiter X-Axis Increment Integer Part Specifies integer part of V limiter x-axis increment.	W

Note: S-TYPE-3, P-TYPE-3

63.2.23 LVYADDI : V Limiter Y-Axis Increment Integer Part Register

Base address: DRW = 0x4044_4000
DRW_NS = 0x5044_4000

Offset address: 0xA8



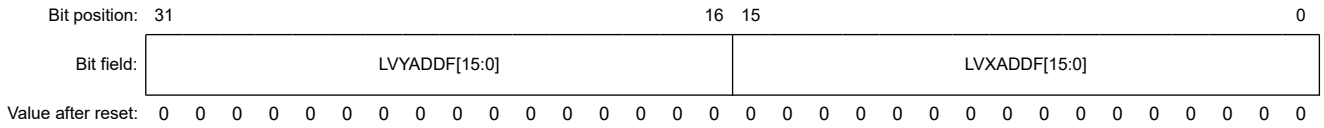
Bit	Symbol	Function	R/W
31:0	LVYADDI[31:0]	V Limiter Y-Axis Increment Integer Part Specifies integer part of V limiter y-axis increment.	W

Note: S-TYPE-3, P-TYPE-3

63.2.24 LVYXADDF : V Limiter Increment Fractional Parts Register

Base address: DRW = 0x4044_4000
DRW_NS = 0x5044_4000

Offset address: 0xAC



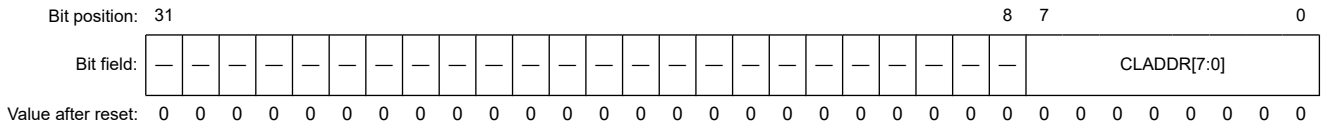
Bit	Symbol	Function	R/W
15:0	LVXADDF[15:0]	V Limiter X-Axis Increment Fractional Part Specifies fractional part of V limiter x-axis increment.	W
31:16	LVYADDF[15:0]	V Limiter Y-Axis Increment Fractional Part Specifies fractional part of V limiter y-axis increment.	W

Note: S-TYPE-3, P-TYPE-3

63.2.25 TEXCLADDR : CLUT Start Address Register

Base address: DRW = 0x4044_4000
DRW_NS = 0x5044_4000

Offset address: 0xDC



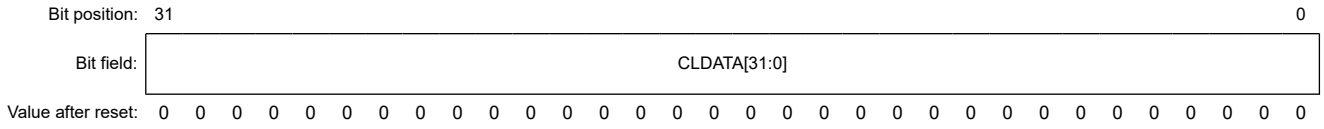
Bit	Symbol	Function	R/W
7:0	CLADDR[7:0]	Texture CLUT Start Address Specifies texture CLUT start address.	W
31:8	—	The write value should be 0.	W

Note: S-TYPE-3, P-TYPE-3

63.2.26 TEXCLDATA : CLUT Data Register

Base address: DRW = 0x4044_4000
DRW_NS = 0x5044_4000

Offset address: 0xE0



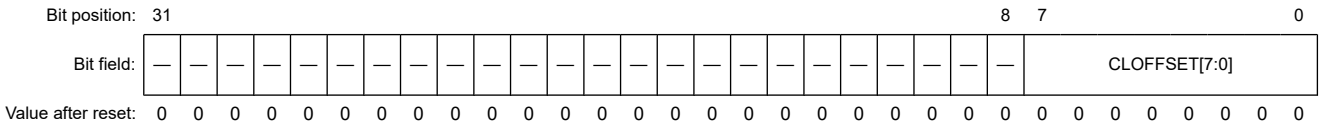
Bit	Symbol	Function	R/W
31:0	CLDATA[31:0]	Texture CLUT Data Specifies texture CLUT data.	W

Note: S-TYPE-3, P-TYPE-3

63.2.27 TEXCLOFFSET : CLUT Offset Register

Base address: DRW = 0x4044_4000
DRW_NS = 0x5044_4000

Offset address: 0xE4



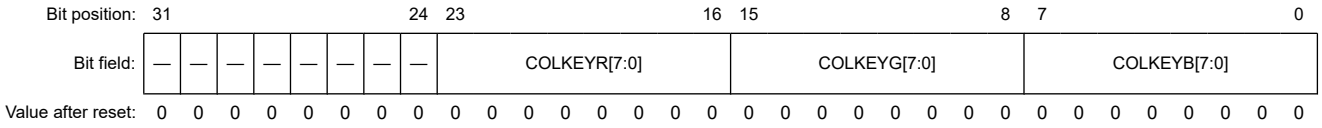
Bit	Symbol	Function	R/W
7:0	CLOFFSET[7:0]	Texture CLUT Offset Specifies texture CLUT offset. CLOFFSET[7:0] is OR gated with the original index.	W
31:8	—	The write value should be 0.	W

Note: S-TYPE-3, P-TYPE-3

63.2.28 COLKEY : Color Key Register

Base address: DRW = 0x4044_4000
DRW_NS = 0x5044_4000

Offset address: 0xE8



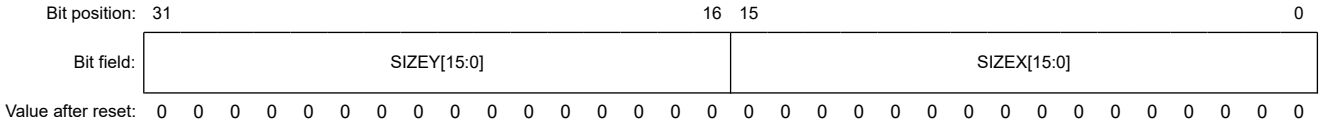
Bit	Symbol	Function	R/W
7:0	COLKEYB[7:0]	Blue Channel of Color Key Specifies blue channel of color key.	W
15:8	COLKEYG[7:0]	Green Channel of Color Key Specifies green channel of color key.	W
23:16	COLKEYR[7:0]	Red Channel of Color Key Specifies red channel of color key.	W
31:24	—	The write value should be 0.	W

Note: S-TYPE-3, P-TYPE-3

63.2.29 SIZE : Bounding Box Dimension Register

Base address: DRW = 0x4044_4000
DRW_NS = 0x5044_4000

Offset address: 0x78



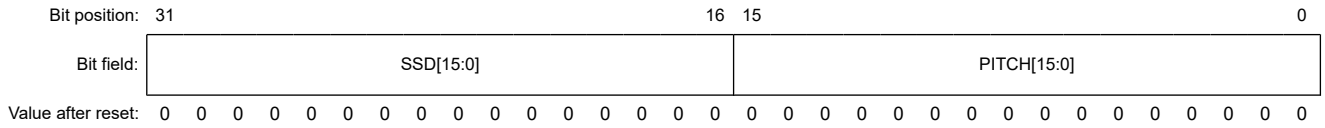
Bit	Symbol	Function	R/W
15:0	SIZEX[15:0]	Bounding Box Width Specifies the width of bounding box in pixels. Valid range is 0 to 1024.	W
31:16	SIZEY[15:0]	Bounding Box Height Specifies the height of bounding box in pixels. Valid range is 0 to 1024.	W

Note: S-TYPE-3, P-TYPE-3

63.2.30 PITCH : Framebuffer Pitch And Spanstore Delay Register

Base address: DRW = 0x4044_4000
DRW_NS = 0x5044_4000

Offset address: 0x7C



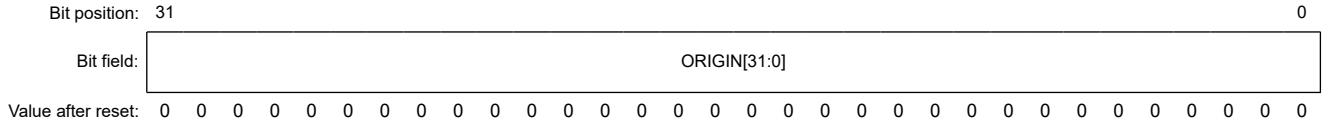
Bit	Symbol	Function	R/W
15:0	PITCH[15:0]	Pitch of the Framebuffer A negative width can be used to render bottom-up instead of top-down.	W
31:16	SSD[15:0]	Spanstore Delay Specifies the number of scan lines to delay spanstore operations.	W

Note: S-TYPE-3, P-TYPE-3

63.2.31 ORIGIN : Framebuffer Base Address Register

Base address: DRW = 0x4044_4000
DRW_NS = 0x5044_4000

Offset address: 0x80



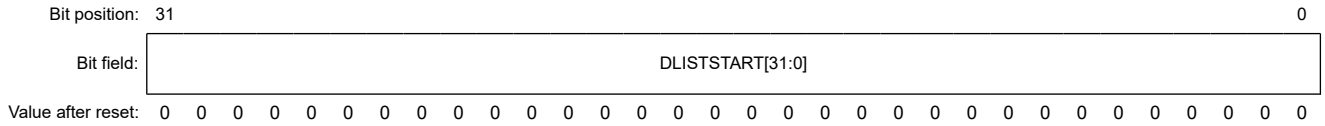
Bit	Symbol	Function	R/W
31:0	ORIGIN[31:0]	Address of the First Pixel in Framebuffer Writing to ORIGIN[31:0] triggers the start of rendering.	W

Note: S-TYPE-3, P-TYPE-3

63.2.32 DLISTSTART : Display List Start Address Register

Base address: DRW = 0x4044_4000
DRW_NS = 0x5044_4000

Offset address: 0xC8



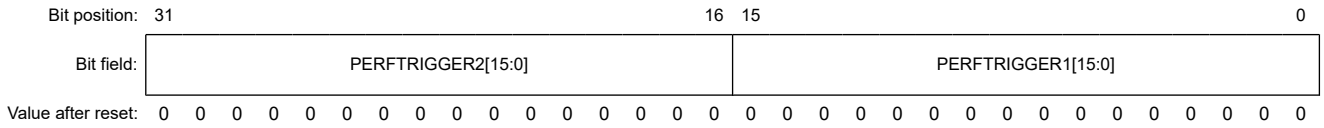
Bit	Symbol	Function	R/W
31:0	DLISTSTART[31:0]	Display List Start Address Setting a new display list base address triggers execution of the new display list. Execution stops only when a new list is set or the current list terminates.	W

Note: S-TYPE-3, P-TYPE-3

63.2.33 PERFTRIGGER : Performance Counters Control Register

Base address: DRW = 0x4044_4000
DRW_NS = 0x5044_4000

Offset address: 0xD4



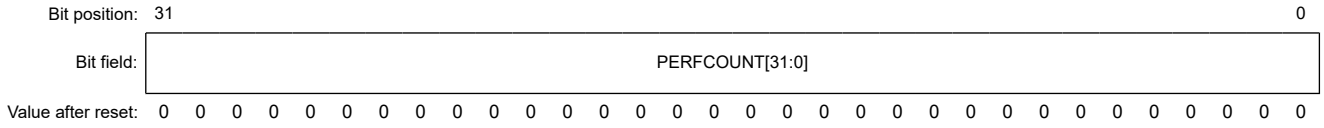
Bit	Symbol	Function	R/W
15:0	PERFTRIGGER1 [15:0]	Trigger of Performance Counter 1 Select the internal event that increments the PERFCOUNT1 register. 0x0000: Disable performance counter 0x0001: Select 2D Drawing Engine active cycles 0x0002: Select framebuffer read access 0x0003: Select framebuffer write access 0x0004: Select texture read access 0x0005: Select invisible pixels (enumerated but selected with alpha 0%) 0x0006: Select invisible pixels while internal FIFO is empty (lost cycles) 0x0007: Select display list reader active cycles 0x0008: Select framebuffer read hits 0x0009: Select framebuffer read misses 0x000A: Select framebuffer write hits 0x000B: Select framebuffer write misses 0x000C: Select texture read hits 0x000D: Select texture read misses 0x001F: Select every clock cycle (for use as timer). Others: Setting prohibited.	W
31:16	PERFTRIGGER2 [15:0]	Trigger of Performance Counter 2 Select the internal event that increments the PERFCOUNT2 register. See the above settings for performance counter 2.	W

Note: S-TYPE-3, P-TYPE-3

63.2.34 PERFCOUNTk : Performance Counter k (k = 1, 2)

Base address: DRW = 0x4044_4000
DRW_NS = 0x5044_4000

Offset address: 0xCC (PERFCOUNT1)
0xD0 (PERFCOUNT2)



Bit	Symbol	Function	R/W
31:0	PERFCOUNT[31:0]	Performance Counter k Value Specifies counter k value. The counter is reset by writing PERFCOUNTk = 0x0000_0000.	R/W

Note: S-TYPE-3, P-TYPE-3

63.2.35 DBWER : DRW Bufferable Write Enable Register

Base address: DRW = 0x4044_4000
DRW_NS = 0x5044_4000

Offset address: 0x100

Bit position:	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Bit field:	—	—	—	—	—	—	—	—	—	—	—	—	—	BWE	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Function	R/W
1:0	—	These bits are read as 0. The write value should be 0.	R/W
2	BWE	Bufferable Write Enable 0: Disables Bufferable Write 1: Enables Bufferable Write	R/W
31:3	—	These bits are read as 0. The write value should be 0.	R/W

Note: S-TYPE-3, P-TYPE-3

63.3 Drawing Features

63.3.1 Drawing Features Summary

63.3.1.1 Color Formats

Supported color formats are:

(1) Framebuffer formats

- 8-bit: A (8)
- 16-bit: RGB (565), ARGB (4444)
- 32-bit: ARGB (8888).

(2) Texture formats

- 1-bit: CLUT (1)/I (1)
- 2-bit: CLUT (2)/I (2)
- 4-bit: CLUT (4)/I (4)
- 8-bit: A (8), CLUT (8)/I (8), ACLUT (44)
- 16-bit: ARGB (4444), ARGB (1555), RGB (565)
- 24-bit: RGB (888) (run length encoded (RLE) unit)
- 32-bit: ARGB (8888).

CLUT formats use a 256-entry color Lookup table.

63.3.1.2 BitBLT Features

The 2D Drawing Engine supports the BitBLT features using its vector drawing function to draw a rectangle and texture it based on the selected BitBLT function. This approach results in the following BitBLT features:

- Fill
- Copy
- Stretch BitBLT
- Rotate and scale
- Alpha blending
- Bilinear filtering
- Color conversion
- Subpixel exact placement.

63.3.1.3 Vector Drawing Features

The vector 2D Drawing Engine uses a half-plane rendering approach, which simplifies implementation of edge antialiasing and blurring features without much overhead. When combining some of its functional units, the module can draw not only linear primitives such as lines or polygons, but also quadratic equation-based primitives such as circles and ellipsoids. The following primitives are supported:

- Lines
- Polygons
- Circles and ellipses
- Quadratic curves (software driver support)
- 2D texture mapping
- Bilinear filtering of the textures.

63.3.2 Vector Drawing

For a detailed explanation of the algorithms, see section 63.6. Rendering Pipeline. Supported vector drawing includes:

(1) Lines

- Arbitrary width
- Round endpoints
- Truncated endpoints
- Alpha gradients
- Soft edges (blurring)
- Render attribute: color, pattern, or texture.

(2) Polygons

- Triangles and quadrangles (complex polygons are tessellated by software)
- Alpha gradients
- Soft edges (blurring)
- Per edge controls for anti-aliasing
- Render attribute: color, pattern, or texture.

(3) Circles and ellipses

- All conic sections
- Filled or with arbitrary width

- Arcs of 0° to 360°
- Soft edges
- Alpha gradients
- Render attribute: color, pattern, or texture.

(4) Quadratic Bézier

- Approximated by circle arcs
- Arbitrary width
- Round or truncated endpoints
- Outlines, blurring
- Alpha gradients
- Render attribute: color.

(5) Texture mapping

- 2D array of pixels that can be mapped implicitly or explicitly on all primitives provided by the 2D Drawing Engine
- Translation, rotation, and scaling/shearing
- Bilinear filtering of the textures
- 3D-like texturing accomplished with line-by-line mapping, if constant in one axis.

63.3.3 BitBLT

A dedicated BitBLT unit is not required in the 2D Drawing Engine. The rendering pipeline described for vector drawing is used as the BitBLT unit and already provides a 1 pixel/cycle throughput. For details, see section 63.6. Rendering Pipeline.

63.3.3.1 Fill

A rectangle in the framebuffer can be filled with any value. Possible color formats are 8-, 16-, or 32-bpp format. The driver optimizes the fill to gain the full benefit of 32-bit parallel rasterization. If the selected color format is less than 32 bpp, the driver corrects the alignment and fills 32 bits per clock, resulting in 2 to 4 times faster fill performance for 8- and 16-bpp formats.

63.3.3.2 Copy

A rectangle in the framebuffer can be filled with any rectangular data from the texture input. When the texture input points to the framebuffer, copying from framebuffer to framebuffer is possible. To avoid copy problems because of overlapping source and destination areas, the copy start point can be selected from top left to bottom right. Possible color formats are 8-, 16-, or 32-bpp format.

The driver optimizes the copy to gain the full benefit of 32-bit parallel rasterization. If the selected color format is less than 32 bpp, the driver corrects the alignment and copies 32 bits per clock, resulting in 2 to 4 times faster copy performance for 8- and 16-bpp formats.

63.3.3.3 Stretch BitBLT

This is similar to the normal copy operation. Because the copy is done as a type of texture mapping, the full texture mapping feature set can be used. Any scaling ratios in the x and y directions is selectable, and filtering can be enabled independently for each axis.

63.3.3.4 Rotate and Scale

This is similar to the normal copy operation. Because the copy is done as a type of texture mapping, the full texture mapping feature set can be used. Any scaling ratios in the x and y direction and any rotation angle is selectable. The x and y directions can be rotated and scaled independently, and filtering for the scalers can be enabled independently for each axis.

63.3.3.5 Alpha Blending

Alpha blending is a fundamental block in the rendering pipeline, so the full alpha blend feature set is available for any BitBLT operation. It is possible to copy an area and blend it over the destination by using any constant global alpha value (register value) or by using an alpha mask. The alpha mask is part of the texture data and can be either a per-pixel value together with a pixel color (ARGB formats) or an alpha-only format using a register color.

In addition to the color channels, the alpha channel can be blended. The formula for the alpha channel can be set independently from the formula for the color channels.

63.3.3.6 Bilinear Filtering

The texture unit can be used to scale, rotate, or shear images. The texturing result can be filtered in the x and y directions independently. When selecting both filters, the result is a bilinear filtered texture. Using the unit twice with two independent textures would generate trilinear filtered bitmaps, improving the visual impression for high dynamic scale ratios.

63.3.3.7 Color Conversion

Color conversion is required when using different texture formats than the framebuffer format. To save texture memory, several formats are supported with less bpp than the framebuffer normally has. The 2D Drawing Engine always operates internally with 32-bpp ARGB (8888). All input data is converted into 32 bpp, and is finally converted back into the framebuffer format.

63.4 Input and Output Data Formats

63.4.1 Source and Destination Data

There are two possible inputs, the framebuffer and the texture or pattern input. The output is always the framebuffer.

Every drawing operation is internally rendered in 32 bpp ARGB (8888). If the input color does not provide an alpha channel, the blue channel is taken as the alpha channel. This alpha can be substituted with any alpha (for example, by an external constant) during the colorization step in the 2D Drawing Engine.

63.4.2 Framebuffer Color Formats

Table 63.2 shows the supported framebuffer color formats.

Table 63.2 Framebuffer color formats

Framebuffer memory occupation	Format	Remarks
8 bpp	A (8)	This color format uses 1 byte per pixel. The alpha channel is internally replicated on the red, blue, and green channels and can be substituted with any color during the color step in the 2D Drawing Engine.
16 bpp	RGB (565)	This color format uses 2 bytes per pixel with 5 bits for red and blue and 6 bits for green. The blue color is taken as the alpha channel during color conversion. The alpha can be substituted with any alpha during the colorization step in the 2D Drawing Engine.
	ARGB (4444)	This color format uses 2 bytes per pixel with 4 bits for each color and alpha channel.
32 bpp	ARGB (8888)	This color format uses 4 bytes per pixel with 8 bits for each color and alpha channel.

The framebuffer color format is selected in the Surface Control Register with the CONTROL2.READFORMAT[2:0] bits.

63.4.3 Texture Color Formats

Table 63.3 shows the supported texture color formats.

Table 63.3 Texture color formats

Texture memory occupation	Format	Remarks
1 bpp	CLUT (1)/I (1)	In this mode, a 1-bit index is used to address one of 256 predefined colors in the color lookup table. If the CLUT is not used, the index is taken as a luminance value.
2 bpp	CLUT (2)/I (2)	In this mode, a 2-bit index is used to address one of 256 predefined colors in the color lookup table. If the CLUT is not used, the index is taken as a luminance value.
4 bpp	CLUT (4)/I (4)	In this mode, a 4-bit index is used to address one of 256 predefined colors in the color lookup table. If the CLUT is not used, the index is taken as a luminance value.
8 bpp	CLUT (8)/I (8)	In this mode, an 8-bit index is used to address one of 256 predefined colors in the color lookup table. If the CLUT is not used, the index is taken as a luminance value.
	A (8)	This color format uses 1 byte per pixel. The alpha channel is internally replicated on the red, blue, and green channels and can be substituted with any color during the colorization step in the 2D Drawing Engine.
	ACLUT (44)	This color format uses 1 byte per pixel. 4 bits are used as an alpha value and 4 bits are used as an index to a color palette. This approach saves space if 16 colors are sufficient to describe the image, because the next bigger alpha format would be 2-byte ARGB (4444).
16 bpp	RGB(565)	This color format uses 2 bytes per pixel with 5 bits for red and blue and 6 bits for green. The blue color is taken as the alpha channel during color conversion. The alpha can be substituted with any alpha during the colorization step in the 2D Drawing Engine.
	ARGB (4444)	This color format uses 2 bytes per pixel with 4 bits for each color and alpha channel.
	ARGB (1555)	This color format uses 2 bytes per pixel. Every color channel has 5 bits and the topmost single bit is taken as an alpha value. This can be used to hold an image with a transparency mask.
24 bpp	RGB (888)	This color format uses 3 bytes per pixel with 8 bits for each color channel. This format is only available as run length encoded data (RLE compression).
32 bpp	ARGB (8888)	This color format uses 4 bytes per pixel with 8 bits for each color and alpha channel.

The texture color format is selected in the Surface Control Register with the CONTROL2.WRITEFORMAT[3:0] bits.

63.5 Texture Data Processing

Figure 63.4 shows the processing of texture data.

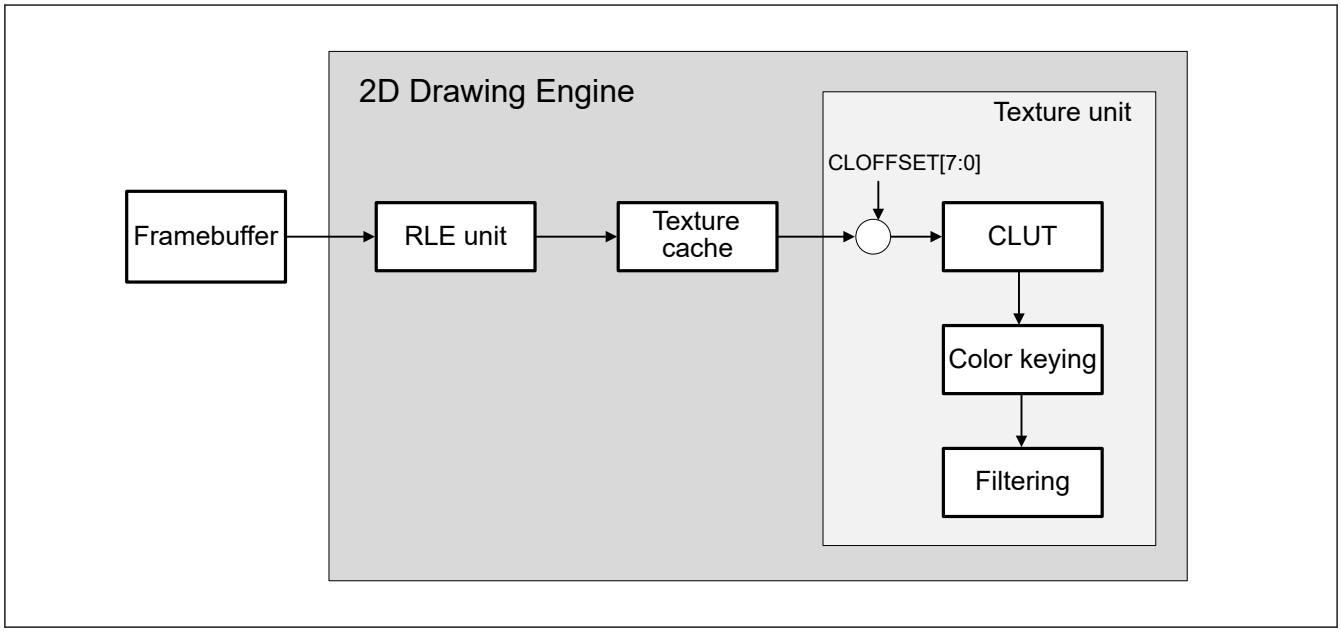


Figure 63.4 Texture data processing

63.5.1 Texture Color Format

Table 63.4 shows the supported texture data formats.

Table 63.4 Texture color formats

Texel bit width	Texel format
32 bits	ARGB (8888)
24 bits	RGB (888)
16 bits	ARGB (4444), ARGB (1555), RGB (565)
8 bits	CLUT (8)I (8), A (8), ACLUT (44)
4 bits	CLUT (4)/I (4)
2 bits	CLUT (2)/I (2)
1 bit	CLUT (1)/I (1)

63.5.2 Run Length Encoded (RLE) Unit

The RLE unit decompresses Targa-like compressed textures and hands the decompressed texel data over to the texture unit. The key features are:

- Support for Targa format
- Avoid the additional Targa limitation to scan lines
- Support for clipping of compressed images, in which the 2D Drawing Engine is allowed to copy only a portion of a larger original texture
- Control of the RLE unit in drawing list operation
- Bypassing of the RLE unit logic if uncompressed textures are fetched from the framebuffer.

(1) Texture cache

The RLE unit feeds the texture cache. The texture cache can be disabled by setting CACHECTL.CENABLETX = 0.

Caution: A texture cache flush operation (CACHECTL.CFLUSHTX = 1) is necessary at the beginning and end of every new RLE texture.

The texture cache and the RLE unit can be bypassed by setting CONTROL2.RLEENABLE = 0.

63.5.2.1 RLE Texel Formats

Table 63.5 lists the data formats supported by the RLE unit.

Table 63.5 Texel formats supported by the RLE unit (1 of 2)

Memory texel format	RLE parameters	RLE coded unit format (CONTROL2.RLEPIXELWIDTH [1:0])	Delivered format
32-bit ARGB (8888)	Included in Targa and RLE formats	32 bits*1 (11b)	32 bits
24-bit RGB (888)		24 bits (10b)	32 bits
16-bit ARGB (4444), ARGB (1555), RGB (565)		16 bits (01b)	16 bits
8-bit CLUT (8)I (8), A (8), ACLUT (44)		8 bits (00b)	8 bits
4-bit CLUT (4)/I (4)	Optional for RLE*2	8 = 2 x 4 bits	4 + 4 bits

Table 63.5 Texel formats supported by the RLE unit (2 of 2)

Memory texel format	RLE parameters	RLE coded unit format (CONTROL2.RLEPIXELWIDTH [1:0])	Delivered format
2-bit CLUT (2)/I (2)	No RLE		
1-bit CLUT (1)/I (1)			

Note 1. 24-bit RGB (888) encoded texels are delivered as ARGB (8888) with Alpha set to 1.
Note 2. Encoding of textures with 4 bits per texel is not defined by the Targa specification but can be done by:

- Combining two 4-bit texels to one byte
- Padding with four 0-bit at the end of the file, if the number of texels is odd
- Encoding as with 8-bit texels.

(1) Texel addressing for RLE textures

The address of a texel is the byte address of the first byte of the texel. The origin of the texture is given by the register TEXORIGIN.

Note: The RLE code must begin at a word boundary of the memory.

Caution: When the FIFO is filled, there is no provision to inhibit read access beyond the end of the RLE code. To avoid memory access violations, the RLE code must be padded by 32 memory words, where every bit of each word is set to 1.

63.5.2.2 Targa RLE Format

Run-length encoded (RLE) images include two types of data elements:

- Run-length packets
- Raw packets.

The first field (1 byte) of each packet is called the repetition count field. The second field is called the pixel value field (1, 2, 3. or 4 bytes). For run-length packets, the pixel value field contains a single pixel value. For raw packets, the field is a variable number of pixel values.

The highest order bit of the repetition count indicates whether the packet is a raw packet or a run-length packet, as follows:

- If bit [7] of the repetition count is set to 1, the packet is a run-length packet
- If bit [7] of the repetition count is set to zero, the packet is a raw packet.

The lower 7 bits of the repetition count specify how many pixel values are represented by the packet. For a run-length packet, this count indicates how many successive pixels have the pixel value specified in the pixel value field. For raw packets, this count specifies how many pixel values are actually contained in the next field. This 7-bit value is actually encoded as 1 less than the number of pixels in the packet (a value of 0 implies 1 pixel while a value of 0x7F implies 128 pixels).

(1) Run-length packet

Run-length packets are composed of two parts. The first is a repetition count and the second is the pixel value to repeat.

Table 63.6 Run-length packet

Field name	Field size
Packet type (must be 1 for run-length)	1 bit
Pixel count (number of pixels encoded in this packet - 1)	7 bits
Pixel data (the shared pixel value to be used)	Pixel depth (field 5.5)

(2) Raw packet

The raw packet always includes two fields. The first field is the repetition count and the second field is the pixel data field.

Table 63.7 Raw packet

Field name	Field size
Packet type (must be 0 for raw packet)	1 bit
Pixel count (number of pixels encoded by this packet - 1)	7 bits
Pixel data	Pixel depth x pixel count - 1

63.5.3 Color Lookup Table (CLUT)

The color lookup table receives an index that addresses one out of the 256 predefined colors.

The predefined color format can be selected as:

- CONTROL2.CLUTFORMAT = 0: ARGB (8888)
- CONTROL2.CLUTFORMAT = 1: RGB (565).

The CLUT is filled by the use of two registers:

- TEXCLDATA
  The ARGB (8888) color definition is written to this register, while the CLUT address is taken from TEXCLADDR.
- TEXCLADDR.
  This is set to the first address of the CLUT to write to and is automatically incremented after each write to TEXCLDATA.

An offset for indexed formats (CLUT (1), CLUT (2), CLUT (4), and CLUT (8)) can be set up in the TEXCLOFFSET register to allow selecting an offset part of the CLUT. The CLUT index is calculated by CLUT (x) or TEXCLOFFSET.CLOFFSET[7:0].

63.5.3.1 CLUT/I Pixel Data Formats

(1) CLUT (1)/I (1) format

The CLUT (1)/I (1) format expresses 1 pixel by using a total of 1 bit.

Memory byte	Pixel
7 (MSB)	P7
6	P6
5	P5
4	P4
3	P3
2	P2
1	P1
0 (LSB)	P0

The left-most pixel is stored at the lowest bit of the memory byte.

(2) CLUT (2)/I (2) format

The CLUT (2)/I (2) format expresses 1 pixel by using a total of 2 bits.

Memory byte	Pixel
7 (MSB)	P3
6	
5	P2
4	
3	P1
2	
1	P0
0 (LSB)	

The left-most pixel is stored at lowest 2 bits of the memory byte.

(3) CLUT (4)/I (4) format

The CLUT (4)/I (4) format expresses 1 pixel by using a total of 4 bits.

Memory byte	Pixel
7 (MSB)	P1
6	
5	
4	
3	P0
2	
1	
0 (LSB)	

The left-most pixel is stored at lowest 4 bits of the memory byte.

63.5.4 Color Keying

The 2D Drawing Engine provides a color keying unit in front of the texture unit. It operates as follows:

1. If enabled, the incoming color is compared with a transparent color, defined by COLKEY.
2. If the value matches, the alpha and color values are set to 0 to mark the color as transparent and handle it as if alpha was pre-multiplied.
3. If the value does not match, then alpha is set to 1.
4. Additional operations such as alpha_in × alpha_const are still possible.

With this approach, an object such as a round icon can be cut out from a rectangular texture and still can be faded by a constant alpha over the background.

63.6 Rendering Pipeline

63.6.1 Coordinate Transformation

Coordinate transformation such as rotation, translation, projection, and scaling must be done on the application side. This is not part of the 2D Drawing Engine hardware or driver. Because all coordinates fed into the 2D Drawing Engine are in fixed point format, these calculations can be made in fixed point format and do not require a floating point unit.

63.6.2 Rasterization

During rasterization, the vector data of the object must be converted to pixel data. To convert the data, the program sets up the edge interpolation hardware, called a limiter, for each edge of the object that calculates a decision value. The limiter

determines which side of the edge the pixel is positioned on. The 2D Drawing Engine includes six internal hardware limiters. In principle, the limiter registers contain the distance between the pixel being processed and the edge.

In the linear setup, a limiter describes a half plane. The intersection of all half planes is the object. If three half planes intersect, a triangle is created as shown in Figure 63.5.

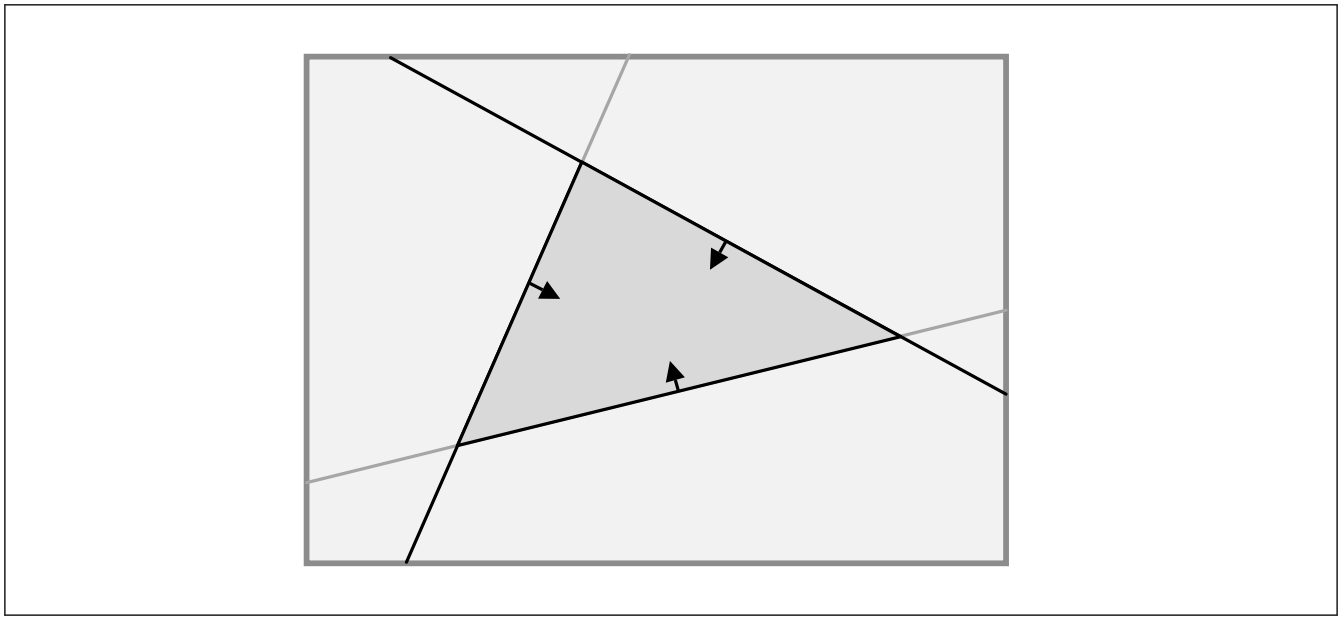


Figure 63.5 Intersection of half planes

The limiter output is clamped to an interval of [0:1]. In limiters 1 and 2 it is possible to apply a band filter before the clamping operation. In this case, the limiter is not describing a half plane but a small band. With this approach, a single limiter can describe a thick line of infinite length.

The output of the different limiters can be combined by the combiner units with a maximum or minimum operation. Maximum operation describes the union of both half planes, and minimum operation describes the intersection of both half planes. The final output is then used as an alpha value. Edge anti-aliasing can be done with no additional effort with this hardware.

To calculate the decision value for each possible pixel with a limiter, the bounding box of the object must be calculated. Then, the decision value for the top left corner of the bounding box must be calculated for each edge. Finally, the increments for a step in the x direction and a step in the y direction must be calculated. This is done by the CPU in the driver.

With this information, the 2D Drawing Engine scans the whole bounding box and calculates the decision value for every pixel incrementally. For a block diagram of the entire rasterization unit, see section 63.1. Overview.

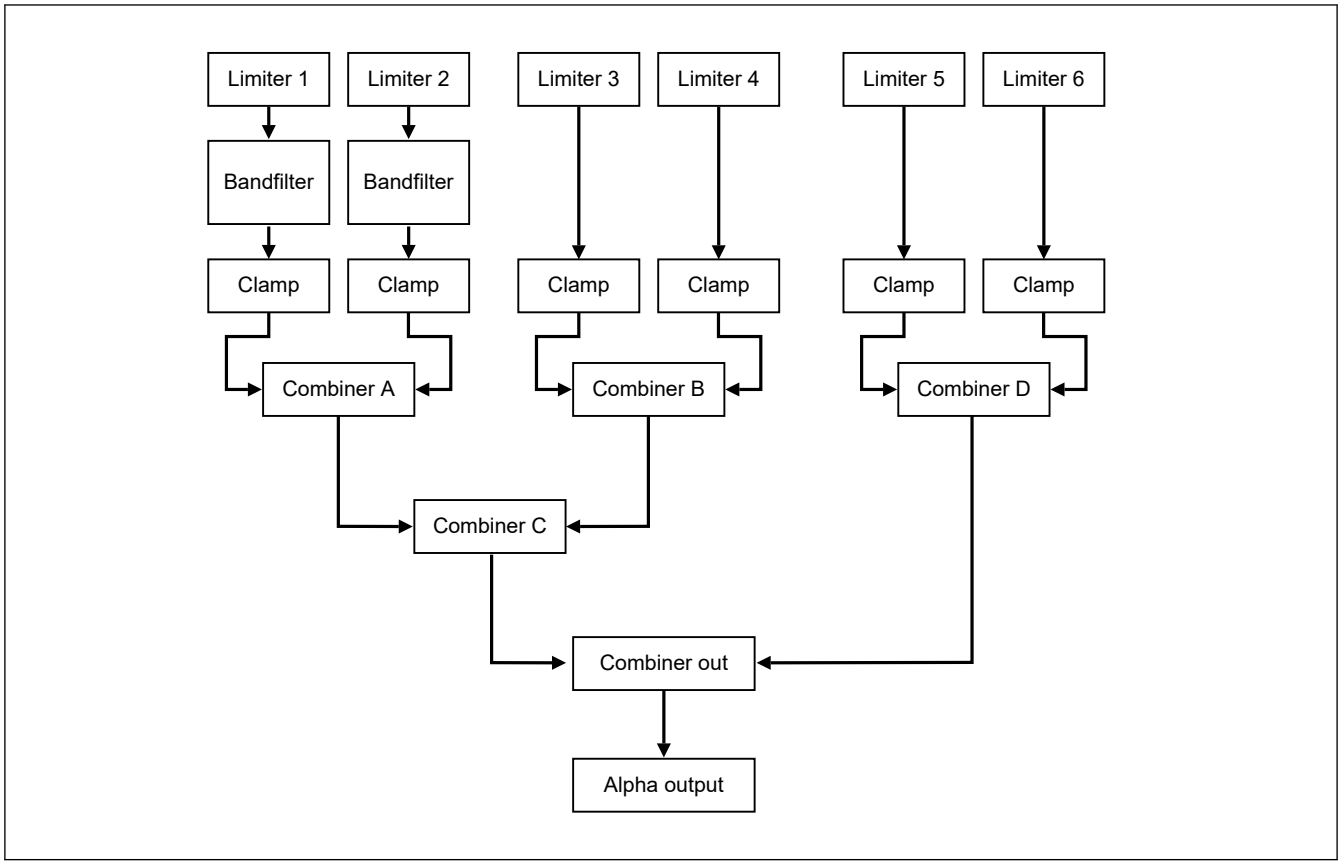


Figure 63.6 Block diagram of rasterization unit

The limiters calculate distances and the combiner units combine them to an alpha value. The combiner units define the conditions for whether or not a pixel is in the bounding box. The alpha value must be greater than 0.

Note: It is possible to have all limiters switched off.

63.6.2.1 Edge Setup Linear Case

(1) Mathematical background

To setup a linear edge, consider the line equation in the classical form.

This can be written as:

$y = \widetilde{a}x + \widetilde{b}$

This can be rewritten as:

$0 = f(x,y) = \widetilde{a}x - y + \widetilde{b} = \mathrm{ax} + \mathrm{by} + c$

with $a = \widetilde{a}, b = -1, c = \widetilde{b}$

This is a more general form. Consider a vector form of this equation:

$\vec{p} = \begin{pmatrix} x \\ y \end{pmatrix}, \vec{n} = \begin{pmatrix} a \\ b \end{pmatrix} \Rightarrow f(x,y) = \mathrm{ax} + \mathrm{by} + c = \vec{p} \cdot \vec{n} + c$

If a point $\vec{p}_0$ is on the line:

$0 = f(x,y) = \vec{p}_0 \cdot \vec{n} + c \Rightarrow c = -\vec{p}_0 \cdot \vec{n}$

Rewriting the constant, the equation becomes:

$c = -\vec{p}_0 \cdot \vec{n} \Rightarrow f(x,y) = \left(\vec{p} - \vec{p}_0\right) \cdot \vec{n}$

The vector $\vec{n}$ is called the normal vector and is perpendicular to the line. The setup can be seen in Figure 63.7.

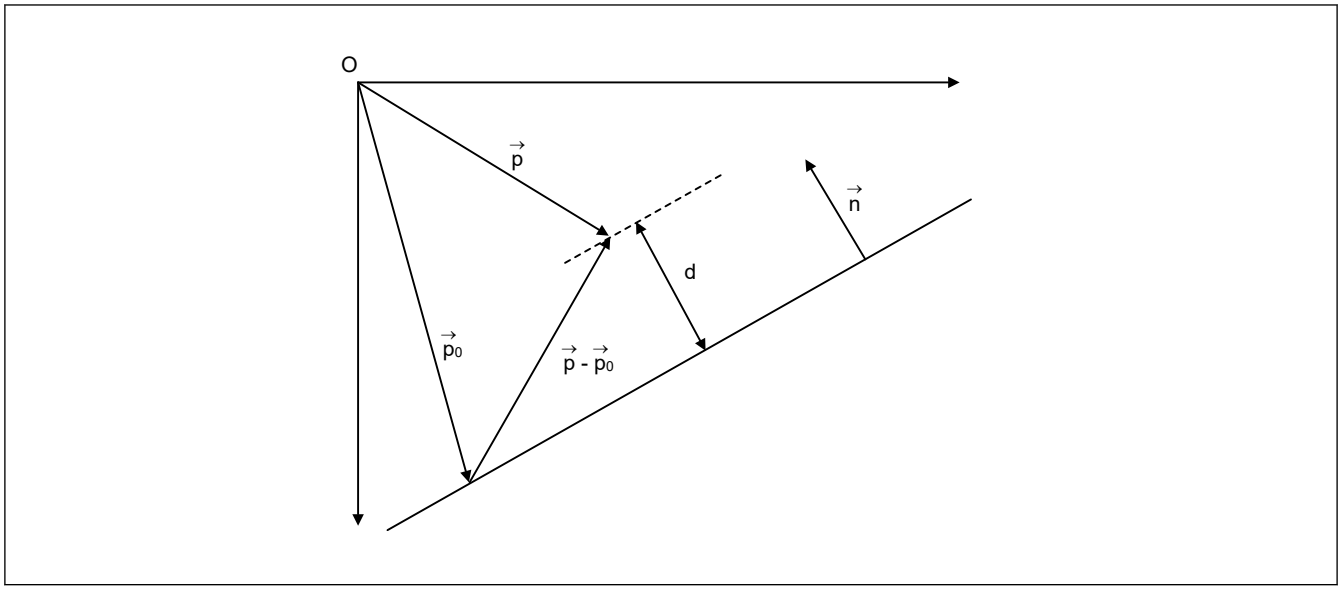


Figure 63.7 Distance of a point to a line

The projection of $\vec{p} - \vec{p}_0$ to $\vec{n}$ is the distance d of the point P to the line. In this case, f (x, y) describes the distance to the line of the pixel with coordinates (x, y).

To calculate the distance of every pixel of the bounding box incrementally, first the distance to the line at origin must be calculated:

$$f(0,0) = -\vec{p}_0 \cdot \vec{n} = c$$

Next the increments for a step in the x direction and a step in the y direction must be calculated:

$$f\left(\vec{p} + \vec{e}_x\right) = \left(\vec{p} + \vec{e}_x - \vec{p}_0\right) \cdot \vec{n} = f\left(\vec{p}\right) + \vec{e}_x \cdot \vec{n} = f\left(\vec{p}\right) + a$$

$$f\left(\vec{p} + \vec{e}_y\right) = \left(\vec{p} + \vec{e}_y - \vec{p}_0\right) \cdot \vec{n} = f\left(\vec{p}\right) + \vec{e}_y \cdot \vec{n} = f\left(\vec{p}\right) + b$$

with $\vec{e}_x = \begin{pmatrix}1\\0\end{pmatrix}, \vec{e}_y = \begin{pmatrix}0\\1\end{pmatrix}$

Consequently, the new distance can be calculated from the old distance with the increments a and b. A step in the x direction changes the distance by a, and a step in the y direction changes the distance by b. The distance of the origin to the line is c.

With this information, the entire bounding box can be scanned. If the bounding box top left corner is not at the origin, an offset must be added.

(2) Limiter operation

The 2D Drawing Engine contains six limiters. Each limiter contains three registers:

- LnSTART
- LnXADD
- LnYADD.

See Figure 63.8 for details. It is possible to drive the limiters in a threshold mode, in which all values above 0.5 are set to 1, and all values below or equal to 0.5 are set to 0. This feature is used when anti-aliasing is not wanted.

Note: In Figure 63.8, the following abbreviations are used:
start = LnSTART
xadd = LnXADD
yadd = LnYADD

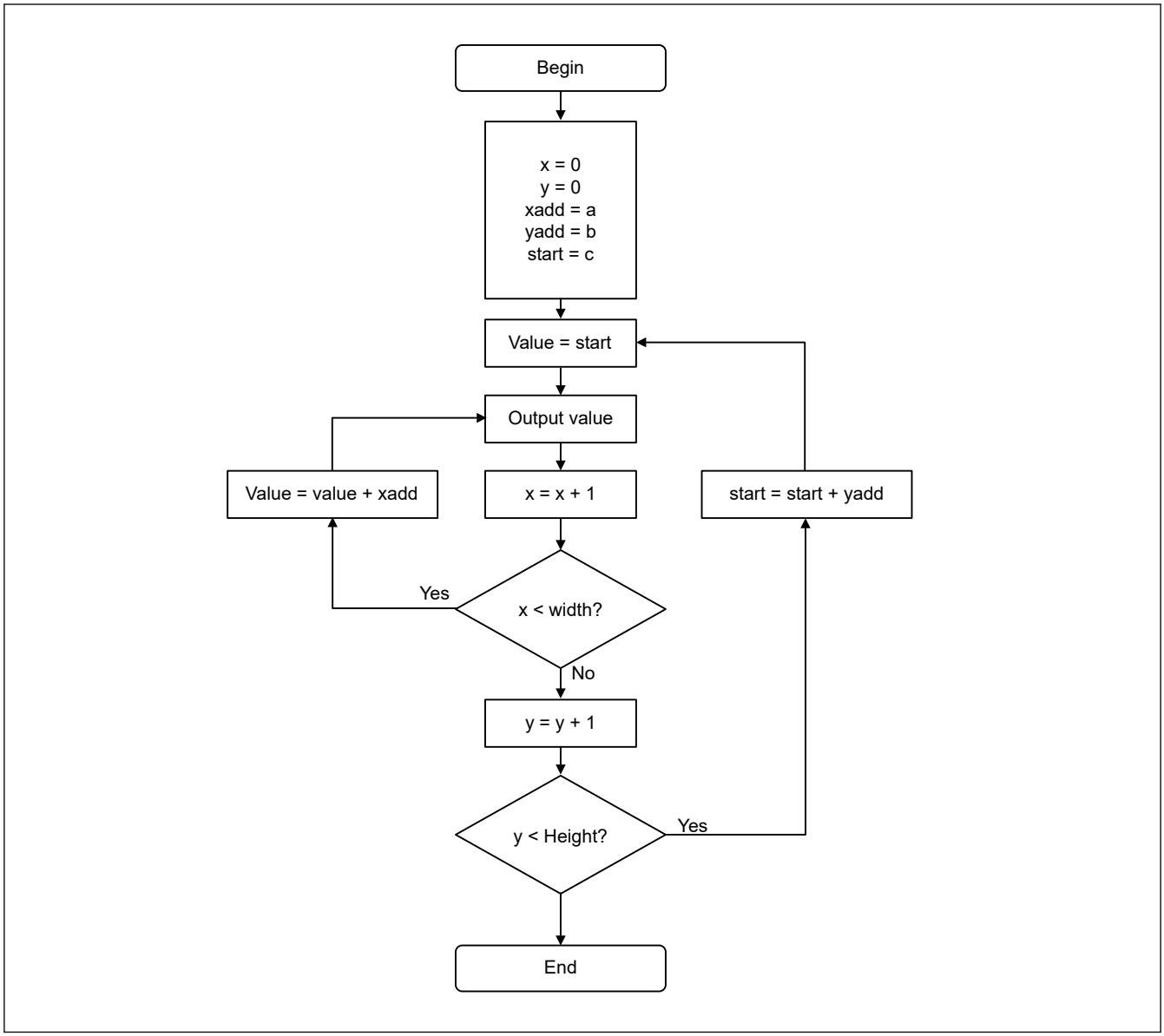


Figure 63.8 Operation flow of the linear limiter

(3) Example

If a straight line is given by the points P0 and P1, then the values are calculated as follows:

$$\Delta\vec{p} = \vec{p}_1 - \vec{p}_0 = \begin{pmatrix} x_1 - x_0 \\ y_1 - y_0 \end{pmatrix} = \begin{pmatrix} \Delta x \\ \Delta y \end{pmatrix}$$

The normal vector (perpendicular but not unit size) is then:

$$\vec{n} = \begin{pmatrix} -\Delta y \\ \Delta x \end{pmatrix}$$

The not normalized distance between edge and origin is then:

$$\vec{p}_0 \cdot \vec{n} = -x_0\Delta y + y_0\Delta x$$

The limiter parameters would be:

$$\text{start} = -x_0\Delta y + y_0\Delta x$$

$$\text{xadd} = -\Delta y$$

$$\text{yadd} = \Delta x$$

In the normalized case, the normal vector is:

$$\overrightarrow{n} = \begin{pmatrix} -\Delta y \\ \Delta x \end{pmatrix} \cdot \frac{1}{\sqrt{\Delta x^2 + \Delta y^2}}$$

The distance between edge and origin is:

$$\overrightarrow{p}_0 \cdot \overrightarrow{n} = \left(-x_0 \cdot \Delta y + y_0 \cdot \Delta x \right) \cdot \frac{1}{\sqrt{\Delta x^2 + \Delta y^2}}$$

The limiter parameters would be:

$$\text{start} = \left(-x_0 \Delta y + y_0 \Delta x \right) \cdot \frac{1}{\sqrt{\Delta x^2 + \Delta y^2}}$$

$$\text{xadd} = -\Delta y \cdot \frac{1}{\sqrt{\Delta x^2 + \Delta y^2}}$$

$$\text{yadd} = \Delta x \cdot \frac{1}{\sqrt{\Delta x^2 + \Delta y^2}}$$

Normalization is only required if anti-aliasing is used. The driver contains an optimized inverse square root function to speed up the normalization process.

63.6.2.2 Edge Setup Quadratic Case

(1) Mathematical background

It is also possible to set up the limiters to incrementally calculate the following equation:

$$f(x, y) = ax^2 + by^2 + cx + dy + f$$

At the origin, the value is:

$$f(0, 0) = f$$

The step in the x direction is:

$$f(x + 1, y)$$

$$= a(x + 1)^2 + by^2 + c(x + 1) + dy + f$$

$$= ax^2 + 2ax + a + by^2 + cx + c + dy + f$$

$$= f(x, y) + 2ax + c + a$$

$$dx(x) = f(x + 1, y) - f(x, y) = 2ax + c + a$$

The step in the y direction is:

$$f(x, y + 1)$$

$$= ax^2 + b(y + 1)^2 + cx + d(y + 1) + f$$

$$= ax^2 + by^2 + 2by + b + cx + d(y + 1) + f$$

$$= f(x, y) + 2by + d + b$$

$$dy(y) = f(x, y + 1) - f(x, y) = 2by + d + b$$

In the quadratic case, the increments dx and dy depend on x and y and are not constant. They can be calculated incrementally:

$$d^2x = dx(x + 1) - dx(x) = 2a(x + 1) + c + a - (2ax + c + a) = 2a$$

$$d^2y = dy(y + 1) - dy(y) = 2b(y + 1) + d + b - (2by + d + b) = 2b$$

At the origin, the increments are:

$dx(0) = c + a$ and $dy(0) = d + b$

By incrementing the value by dx and dy for every step in the x and y direction and incrementing dx, dy by d^2x, and d^2y for every step in the x and y direction, the quadratic equation can be easily calculated for the whole bounding box.

(2) Limiter operation

In the quadratic case, two linear limiters are combined to operate as one quadratic limiter, called limiter 1 and limiter 2. The registers are:

- L1START, L1XADD, L1YADD
- L2START, L2XADD, L2YADD.

See Figure 63.9 for details. The gray box is an addition that performs a different operation, as in the linear setup. It is possible to drive the limiters in a threshold mode, in which all values above 0.5 are set to 1, and all values below or equal to 0.5 are set to 0. This feature is used when anti-aliasing is not needed.

Note: In Figure 63.9, the following abbreviations are used:

- start1 = L1START, start2 = L2START
- xadd1 = L1XADD, xadd2 = L2XADD
- yadd1 = L1YADD, yadd2 = L2YADD.

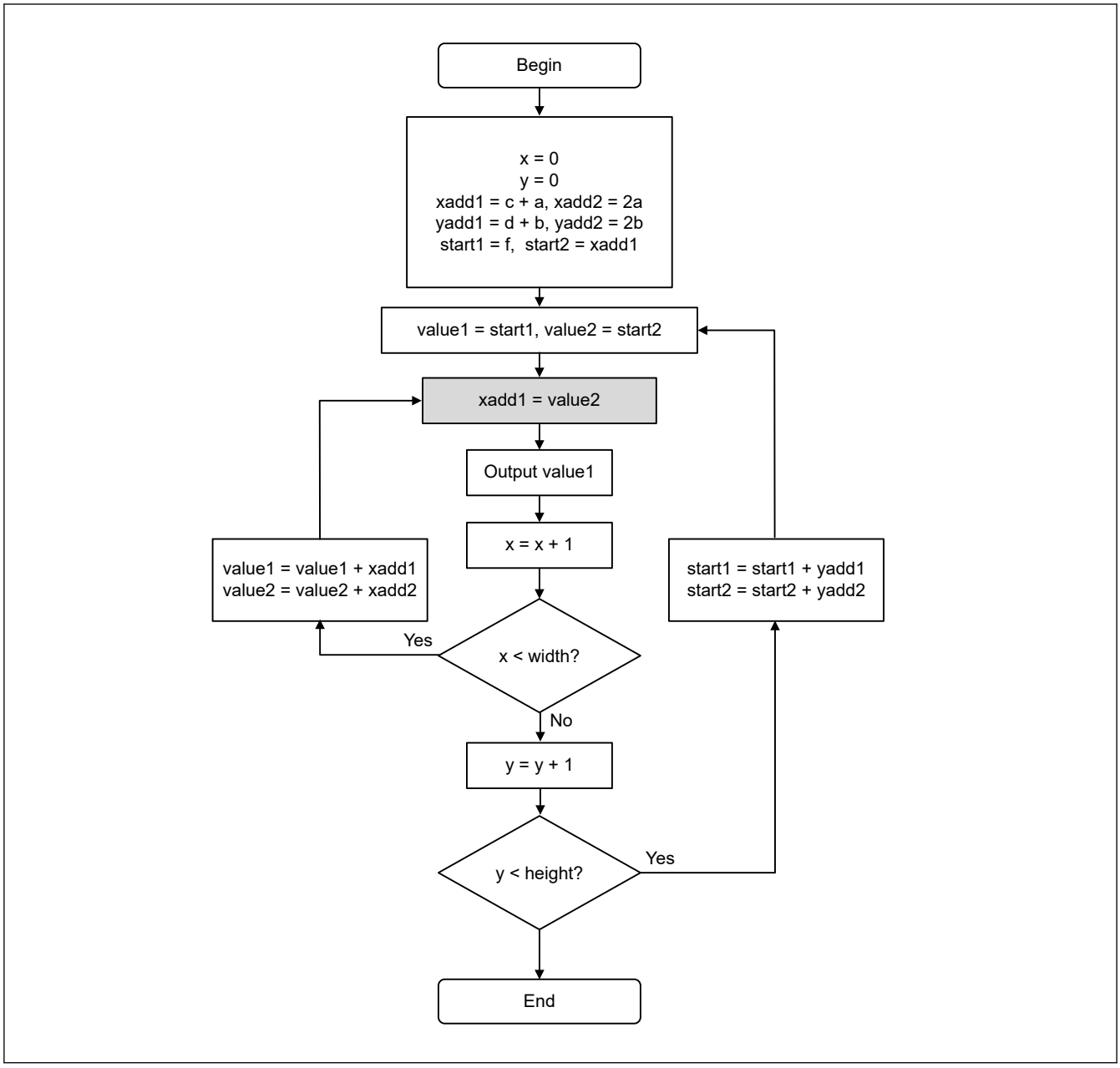


Figure 63.9 Operation flow of the quadratic limiter

(3) Example

Consider the equation for a circle with the center at $\vec{c} = \begin{pmatrix} s \\ t \end{pmatrix}$ and radius r:

$0 = f(x,y) = (x-s)^2 + (y-t)^2 - r^2$

This equation can be rewritten as:

$f(x,y) = x^2 - 2\mathrm{xs} + s^2 + y^2 - 2\mathrm{yt} + t^2 - r^2$

This can be sorted to fit to the original equation:

$f(x,y) = x^2 + y^2 - 2\mathrm{sx} - 2\mathrm{ty} + \left(s^2 + t^2 - r^2\right)$

With the following assignments, the circle equation can be calculated incrementally:

$a = 1$

$b = 1$

$c = -2s$

$d = -2t$

$f = s^2 + t^2 - r^2$

For the limiters with the results calculated in (1) Mathematical background, this would result in:

$\text{start1} = f = s^2 + t^2 - r^2$

$\text{xadd1} = c + a = -2s + 1$

$\text{yadd1} = d + b = -2t + 1$

$\text{start2} = \text{xadd1}$

$\text{xadd2} = 2a = 2$

$\text{yadd2} = 2b = 2$

63.6.2.3 Band Filter

The output of limiter 1 and 2 can be modified to use a band filter. The band filter has a single filter parameter w.

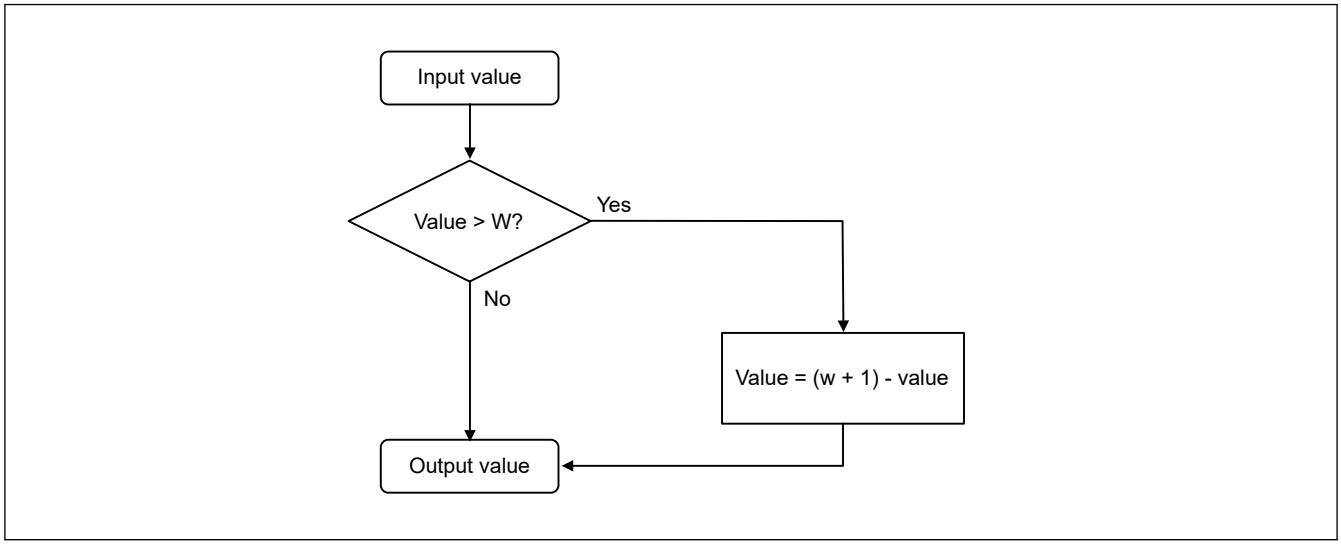


Figure 63.10 Band filter

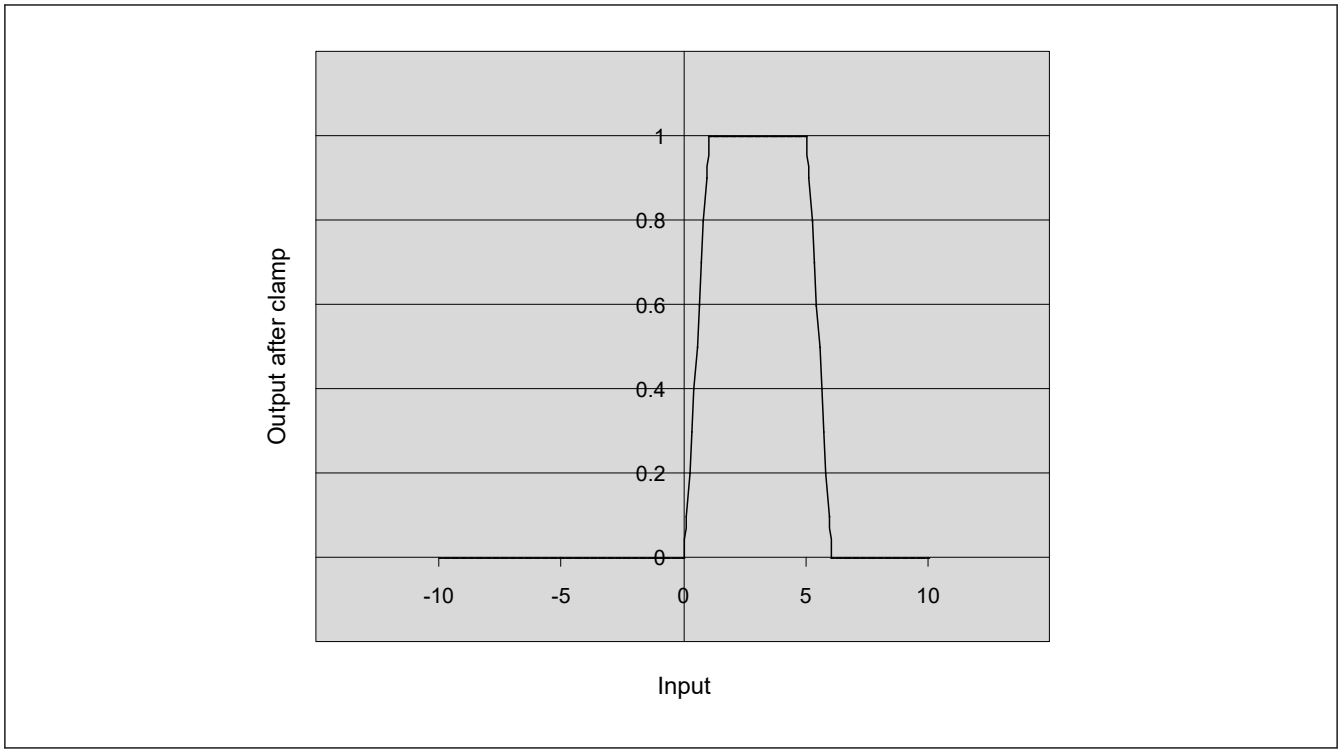


Figure 63.11 Band filter output after clamp with w = 7

63.6.2.4 Clamping Unit

The clamping unit cuts the limiter output to the interval [0:1].

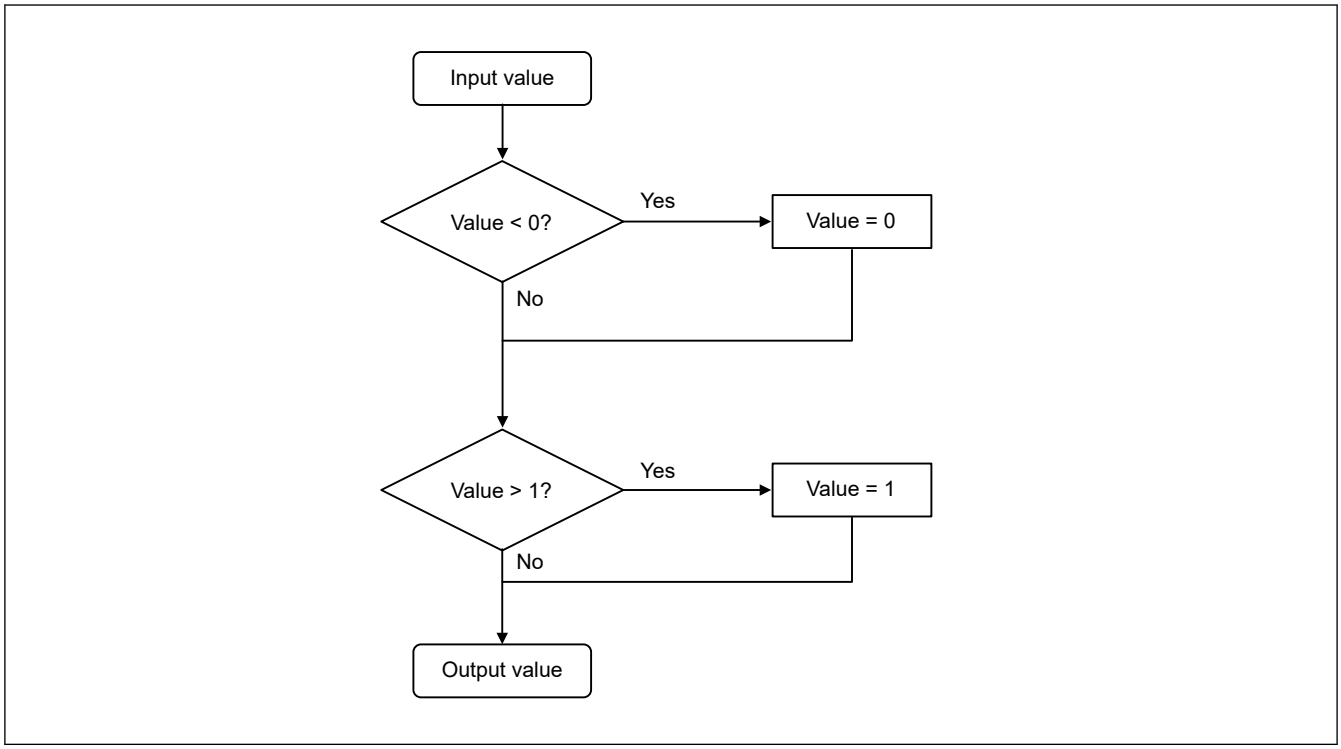


Figure 63.12 Clamping unit

The clamping unit can be put into threshold mode, in which all values greater than 0.5 are set to 1, and all values below or equal to 0.5 are set to 0. This feature is used when anti-aliasing is not needed, such as for shared edges.

63.6.2.5 Combiner Unit

The combiner unit can be operated in minimum mode and in maximum mode. In minimum mode the smaller value is output, and in maximum mode the larger value is output. The minimum mode represents the intersection, and the maximum mode represents the union of the two regions.

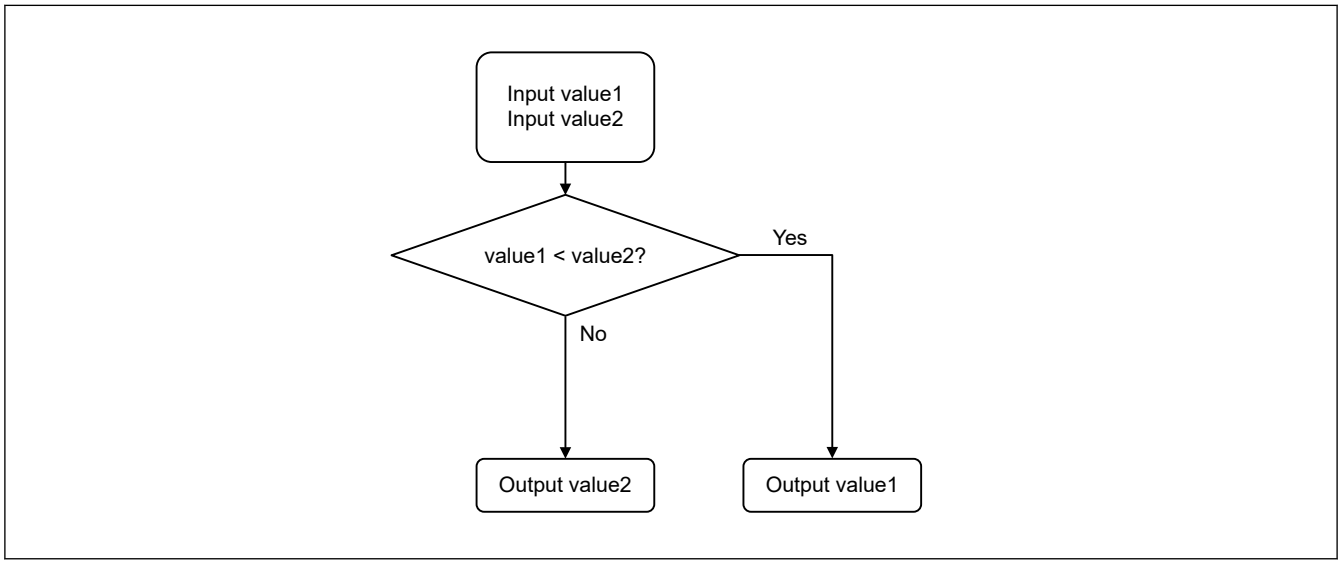


Figure 63.13 Combiner operated in minimum mode with intersection

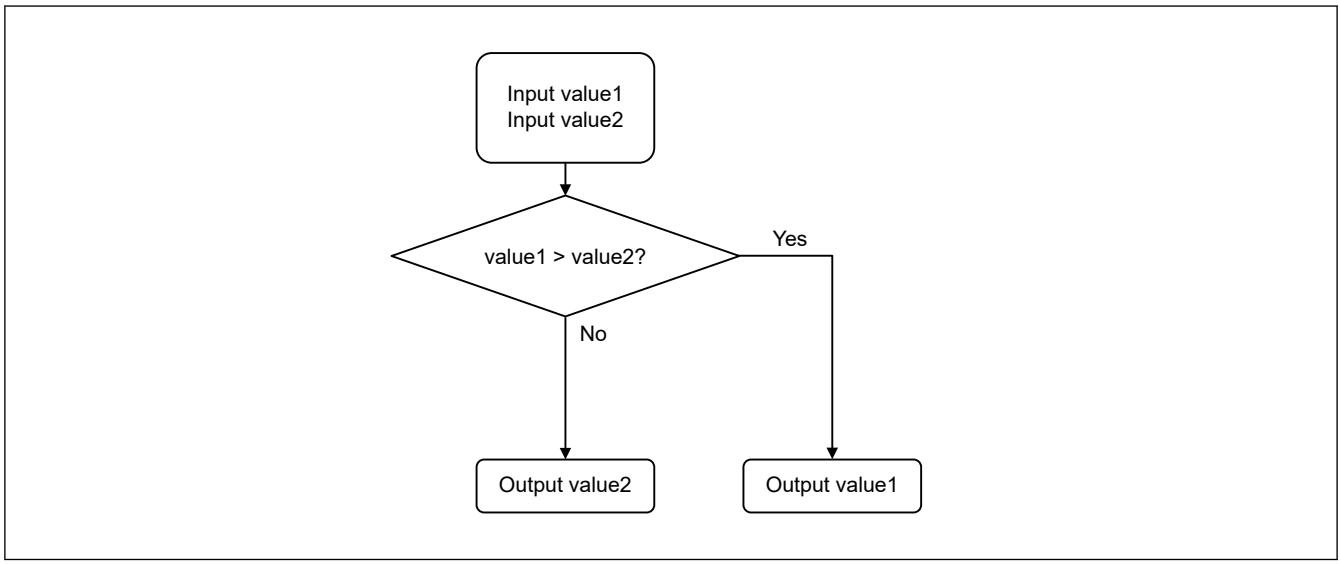


Figure 63.14 Combiner operated in maximum mode with union

63.6.2.6 Rasterization Optimization

During rasterization, it is necessary to step through the whole bounding box one pixel at a time. This requirement can lead to an unnecessary number of steps for pixels that are not drawn. The 2D Drawing Engine provides optimization methods designed to reduce the number of steps required during rasterization. One optimization relies on the fact that any convex primitive can have only one span per line (a span is a contiguous horizontal line). This form of optimization detects a span start and saves the information for the next line. Another optimization is to detect a span end and stop rasterization for the current line.

(1) Spanstore

Consider the case in Figure 63.15.

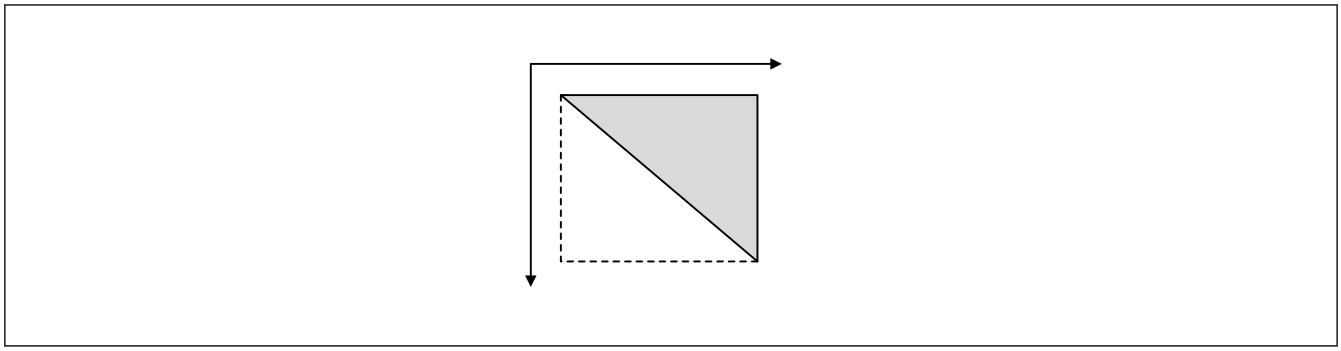


Figure 63.15 Triangle where the first edge is monotone growing

If the gray triangle must be rendered, half of the pixels processed by the rendering engine in the dotted bounding box would not be drawn. This situation can be optimized with the spanstore operation. When spanstore is activated and a span start is detected, the x position of the span start is detected.

In the next line the rendering starts with the stored x position. This only works if the edge is monotonically increasing ($y1 > y2 \Rightarrow x1 \geq x2$).

Consider the case in Figure 63.16.

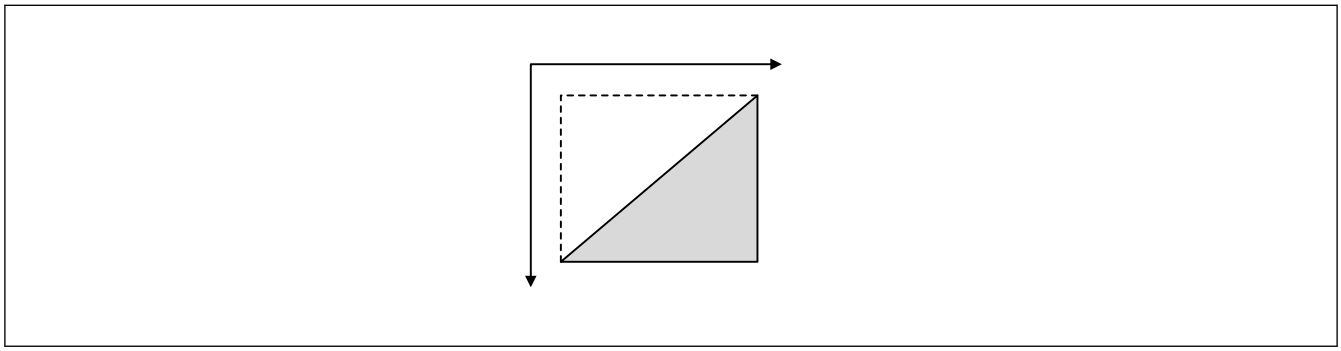


Figure 63.16 Triangle where the first edge is monotone falling

In this case, the normal spanstore operation cannot be performed. For this, the y direction of the rendering is reversed, and spanstore can operate again.

Consider the last case in Figure 63.17.

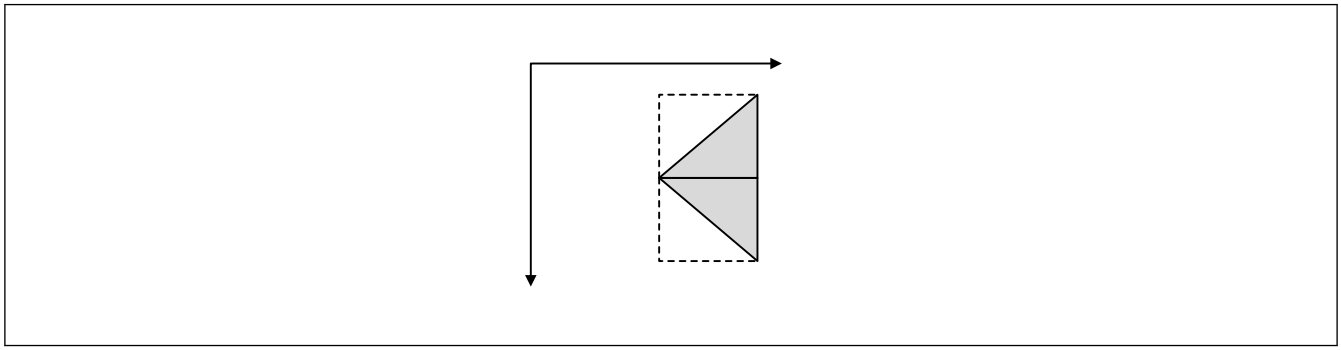


Figure 63.17 Triangle where the first edge is first monotone falling and then monotone growing

In this case, the triangle must be split and rendered as two parts for the spanstore optimization to work.

It is also possible to delay spanstore activation for a number of lines. This approach is used for rasterizing circles, as shown in Figure 63.18.

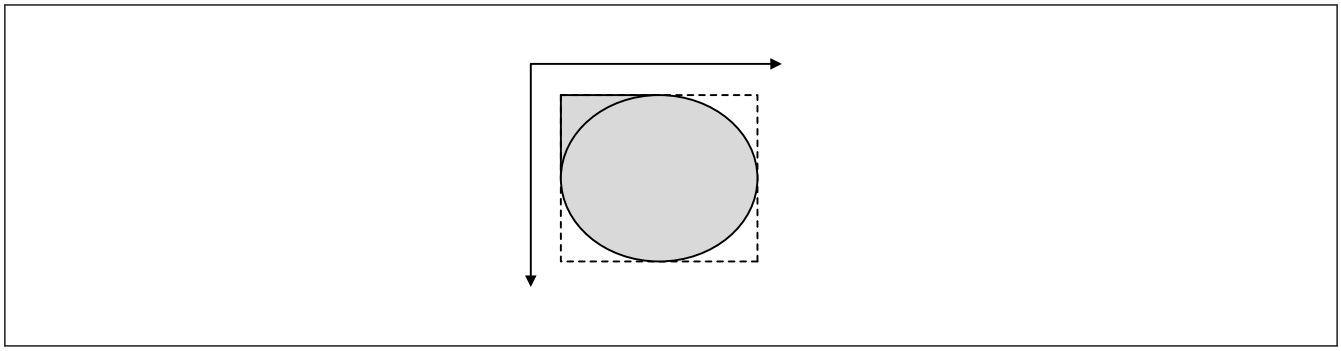


Figure 63.18 Full circle where the first edge is monotone falling for the first half and monotone growing for the second half

In this case, spanstore cannot be activated in the top left corner but can be activated in the bottom left corner. The empty corners in the top right and the bottom right cannot be rasterized because of the spanabort optimization.

(2) Spanabort

The second optimization assumes that the object that must be drawn is convex, which means there is only one span per scan line to be drawn. A non-convex object includes an object such as a triangle that is not filled and only consists of a thick border.

For a convex object, the rasterization can be stopped when the end of the first span is detected. No other constraints apply to this optimization for convex objects.

(3) Optimization efficiency

The efficiency of the optimizations can be seen for a typical case in Figure 63.19. In this case, the triangle is always rendered as single piece and is not separated into multiple triangles for higher optimizations. For this, the spanstore delay is used.

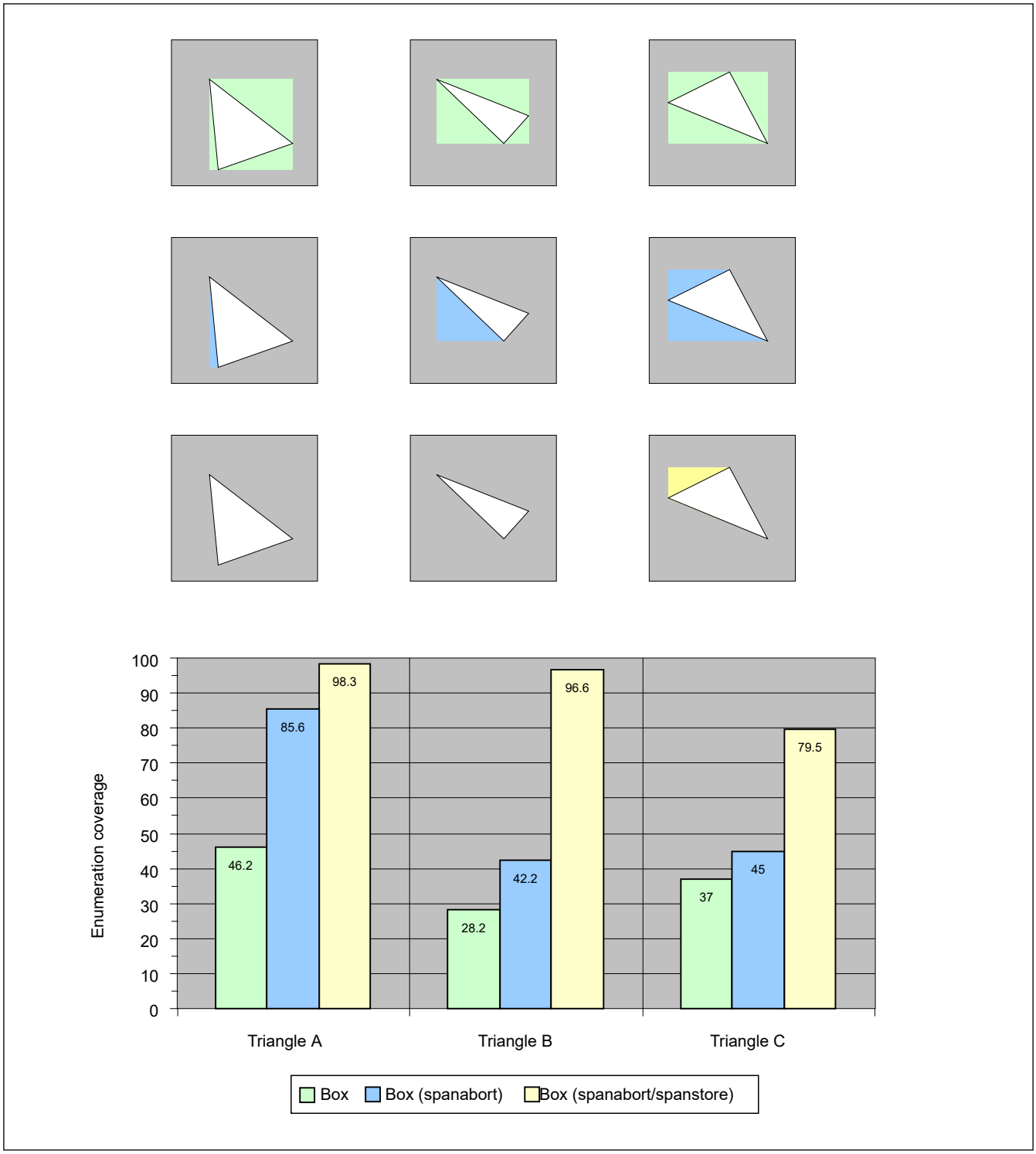


Figure 63.19 Efficiency of spanstore and spanabort optimizations with enumeration coverage equal to pixels of primitive/pixels of bounding box

63.6.3 Texturing

The texture unit can cover any primitive with a picture. The picture can be stretched, sheared, rotated, and translated in one step. To avoid aliasing, the result can be filtered bilinear in the u and v directions.

63.6.3.1 Mathematical Background

The arbitrary mapping problem is completely determined by a mapping from 3 points in the object space (x, y) to 3 points in the texture space (u, v).

Consider the following mapping:

$$\vec{p}_0 = \begin{pmatrix} x_0 \\ y_0 \end{pmatrix} \Rightarrow \left(\widetilde{\vec{p}_0}\right) = \begin{pmatrix} u_0 \\ v_0 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

$$\vec{p}_1 = \begin{pmatrix} x_1 \\ y_1 \end{pmatrix} \Rightarrow \left(\widetilde{\vec{p}_1}\right) = \begin{pmatrix} u_1 \\ v_1 \end{pmatrix} = \begin{pmatrix} w \\ 0 \end{pmatrix}$$

$$\vec{p}_2 = \begin{pmatrix} x_2 \\ y_2 \end{pmatrix} \Rightarrow \left(\widetilde{\vec{p}_2}\right) = \begin{pmatrix} u_2 \\ v_2 \end{pmatrix} = \begin{pmatrix} 0 \\ h \end{pmatrix}$$

where w is the width of the texture and h is the height of the texture.

Examine Figure 63.20 in the object space. To simplify calculations the difference vectors are taken as calculations:

$$\vec{d}_1 = \vec{p}_1 - \vec{p}_0$$

$$\vec{d}_2 = \vec{p}_2 - \vec{p}_0$$

This is equivalent to transforming from coordinate system O to coordinate system O'.

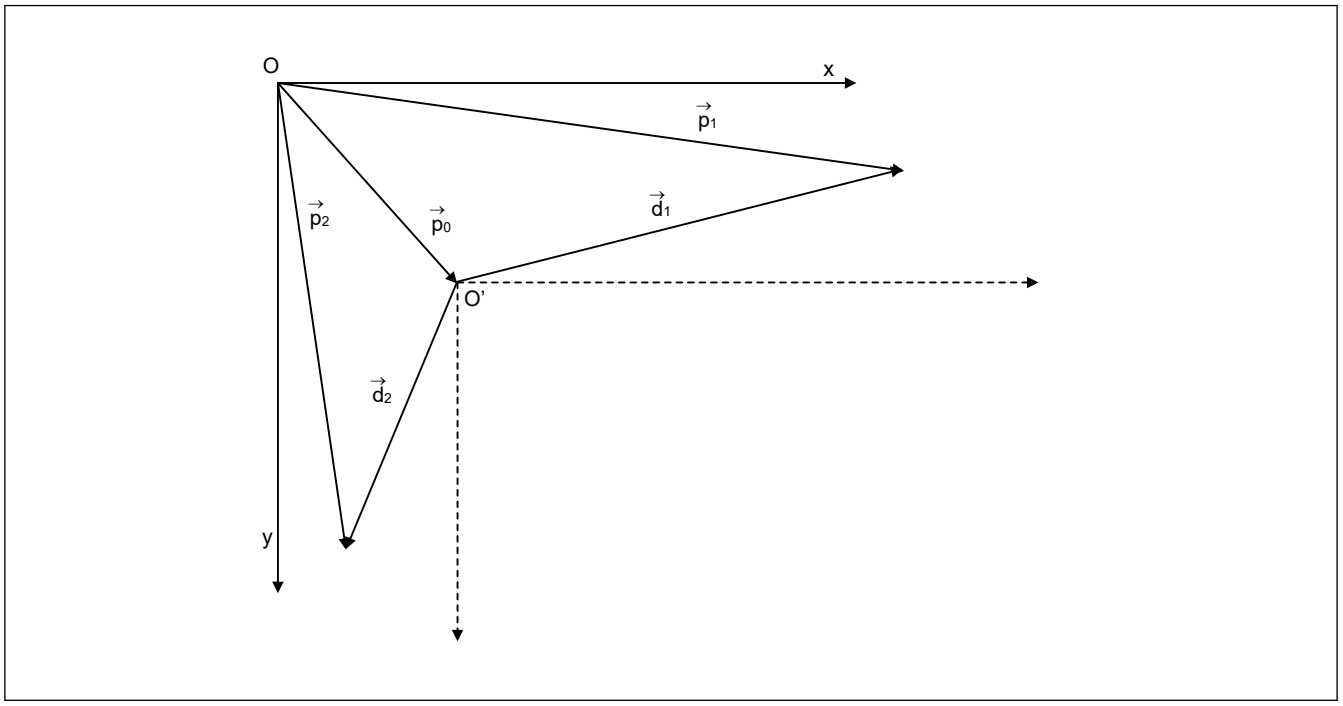


Figure 63.20 Texture mapping, object space, and transformation from coordinate system O to O' to simplify calculations

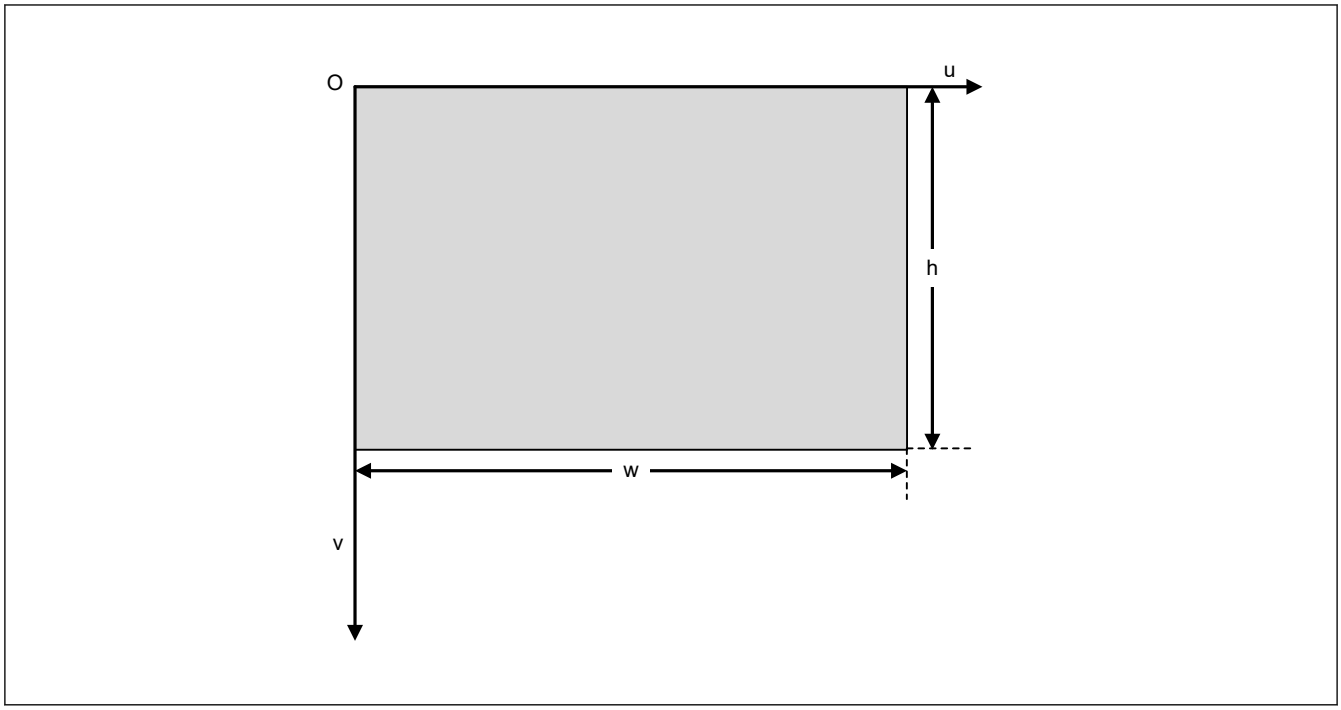


Figure 63.21 Texture mapping, texture space, and texture with width w and height h

In O', the mapping is:

$$\overrightarrow{\mathrm{p}'}_0 = \begin{pmatrix}0\\0\end{pmatrix} \Rightarrow \left(\widetilde{\overrightarrow{p}}_0\right) = \begin{pmatrix}u_0\\v_0\end{pmatrix} = \begin{pmatrix}0\\0\end{pmatrix}$$

$$\overrightarrow{d}_1 = \begin{pmatrix}\mathrm{dx}_1\\\mathrm{dy}_1\end{pmatrix} \Rightarrow \left(\widetilde{\overrightarrow{p}}_1\right) = \begin{pmatrix}u_1\\v_1\end{pmatrix} = \begin{pmatrix}w\\0\end{pmatrix}$$

$$\overrightarrow{d}_2 = \begin{pmatrix}\mathrm{dx}_2\\\mathrm{dy}_2\end{pmatrix} \Rightarrow \left(\widetilde{\overrightarrow{p}}_2\right) = \begin{pmatrix}u_2\\v_2\end{pmatrix} = \begin{pmatrix}0\\h\end{pmatrix}$$

This is a linear mapping that can be described by a 2 x 2 matrix.

$$\left(\widetilde{\overrightarrow{p}}\right) = M \cdot \overrightarrow{\mathrm{p}'} = \begin{bmatrix}m_{11} & m_{12}\\m_{21} & m_{22}\end{bmatrix} \cdot \overrightarrow{\mathrm{p}'}$$

$$\Rightarrow \begin{pmatrix}\mathrm{w}\\0\end{pmatrix} = M \cdot \overrightarrow{d}_1 \wedge \begin{pmatrix}0\\h\end{pmatrix} = M \cdot \overrightarrow{d}_2$$

If the equations are expanded and sorted, the result is two equation systems each with two unknowns. These can be described more easily with a new matrix.

Let A = $\begin{bmatrix}\mathrm{dx}_1 & \mathrm{dy}_1\\\mathrm{dx}_2 & \mathrm{dy}_2\end{bmatrix}$ then the equation can be rewritten as:

$$\begin{pmatrix}\mathrm{w}\\0\end{pmatrix} = A \cdot \begin{pmatrix}m_{11}\\m_{12}\end{pmatrix} \wedge \begin{pmatrix}0\\h\end{pmatrix} = A \cdot \begin{pmatrix}m_{21}\\m_{22}\end{pmatrix}$$

This can be easily solved with determinants.

Let $\mathrm{c} = \frac{1}{\mathrm{detA}} = \frac{1}{\mathrm{dx}_1 \cdot \mathrm{dy}_2 - \mathrm{dx}_2 \cdot \mathrm{dy}_1}$

The resulting constants are:

$$m_{11} = c \cdot w \cdot \mathrm{dy}_2 = \frac{\mathrm{du}}{\mathrm{dx}}$$

$$m_{12} = -\mathrm{c} \cdot w \cdot \mathrm{dx}_2 = \frac{\mathrm{du}}{\mathrm{dy}}$$

$m_{21} = -\mathrm{c} \cdot h \cdot \mathrm{dy}_1 = \frac{\mathrm{dv}}{\mathrm{dx}}$

$m_{22} = c \cdot h \cdot \mathrm{dx}_1 = \frac{\mathrm{dv}}{\mathrm{dy}}$

To calculate the start values for u and v at the top of the bounding box, the transformation from O' to O must be reversed. Let U_s and V_s be the start values, then:

$$\begin{pmatrix} U_s \\ V_s \end{pmatrix} = M \cdot \left(-\vec{p}_0 \right) = c \cdot \begin{pmatrix} -\mathrm{w} \cdot (x_0 \cdot \mathrm{dy}_2 - y_0 \cdot \mathrm{dx}_2) \\ h \cdot (x_0 \cdot \mathrm{dy}_1 - y_0 \cdot \mathrm{dx}_1) \end{pmatrix}$$

(1) Examples

U and v are in the texture space. Enter the following into any case:

- Copy case
  $\mathrm{dx}_1 = 1, \mathrm{dx}_2 = 0, \mathrm{dy}_1 = 0, \mathrm{dy}_2 = 1$
- Scaling case x scaling copy case
  $\mathrm{dx}_1 = f, \mathrm{dx}_2 = 0, \mathrm{dy}_1 = 0, \mathrm{dy}_2 = 1$
  with f being the scaling factor in the x direction similar for the y direction.
- Rotation case
  $\mathrm{dx}_1 = \mathrm{cosa}, \mathrm{dx}_2 = -\mathrm{sina}, \mathrm{dy}_1 = \mathrm{sina}, \mathrm{dy}_2 = \mathrm{cosa}$
  with the angle between d1 and the x axis in the clockwise direction.

63.6.3.2 Limiter Operation

The texture limiters operate exactly the same as the spatial limiters shown in Figure 63.8

The register layout for the u limiter is the same.

- LUSTART = U_s
- LUXADD = du/dx
- LUYADD = du/dy.

The register layout for the v limiter is slightly different. TEXPITCH is multiplied to save one hardware multiplier.

- LVSTARTI = floor (V_s) • TEXPITCH
  Contains the integer part of the start value.
- LVSTARTF = (V_s - floor (V_s)) • TEXPITCH
  Contains the fractional part of the start value.
- LVXADDI = floor (dv/dx) • TEXPITCH
  Contains the integer part of dv/dz.
- LVYADD = floor (dv/dy) • TEXPITCH
  Contains the integer part of dv/dy.
- LVYXADDF = (dv/dy - floor (dv/dy)) • TEXPITCH ((dv/dx - floor (dv/dx)) • TEXPITCH)
  Contains the fractional part of dv/dy and dv/dx combined in one register.
- TEXMASK
  Contains a mask for u and v separately to wrap around values of u and v. This is useful for staying inside the limits of the texture or repeat a texture. Wrap around textures have to have a size multiple of 2.
- TEXPITCH
  Contains the width of the texture in pixels in framebuffer memory. This information is required to calculate the new address if stepping to a new line.
- TEXORIGIN
  Contains the base address of the texture.

63.6.4 Colorization

After a pixel is found to be part of the geometry, its color is calculated. The 2D Drawing Engine supports a very general color calculation scheme, allowing support for several color modes. This color scheme uses an interpolation between two color registers, COLOR1 and COLOR2. See Figure 63.22 for details. COLOR1 and COLOR2 are marked as A, B in the figure for clarity.

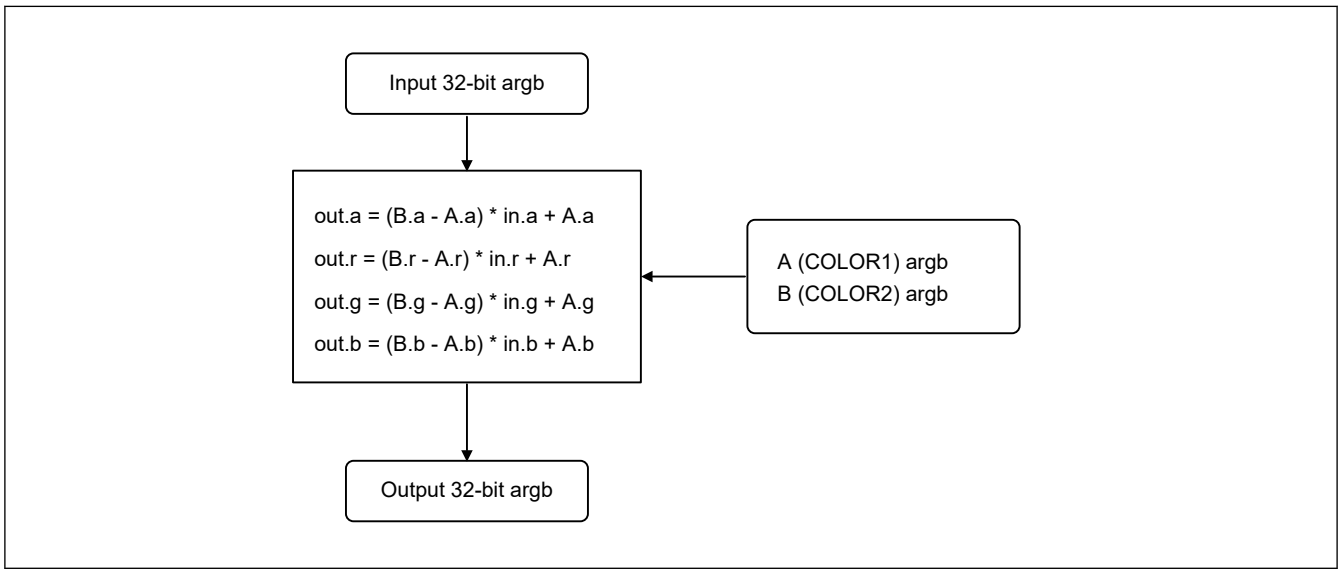


Figure 63.22 Colorization step and interpolation between the two color registers, A (COLOR1) and B (COLOR2)

This general approach can be used to support several different color modes that can be individually applied to any color or alpha channel of the input.

Table 63.8 Colorization operations

Operation	Settings for A and B*1
Copy	A = 0, B = 0xff
Replace with a constant value v	A = v, B = v
Multiply by a constant value v	A = 0, B = v
Colorize an alpha texture with the RGB value v	A.a = 0, A.r = B.r, A.g = B.g, A.b = B.b, B.a = 0xff, B.r = v.r, B.g = v.g, B.b = v.b
Invert a channel	A = 0xff, B = 0
Invert multiply with v	A = v, B = 0
Interpolate between color v and color u	A = v, B = u

Note 1. A = COLOR1, B = COLOR2

63.6.5 Blending

63.6.5.1 Color Channel Blending

The last step before the pixel is written to the framebuffer is to blend the pixel with the data that is already written to the framebuffer. If blending is activated, the framebuffer, referred to as DST, must be read. SRC is the output from the colorization unit.

The following color blend modes are supported:

- SRC_ZERO
- SRC_ONE
- SRC_ALPHA

- SRC_ONE_MINUS_ALPHA
- DST_ZERO
- DST_ONE
- DST_ALPHA
- DST_ONE_MINUS_ALPHA.

The selection of the color channel blend modes is performed with the following flags:

- BSF: blend source factor is alpha
- BSI: blend source factor invert
- BDF: blend destination factor is alpha
- BDI: blend destination factor invert.

The formula for the blending is:

$dst = src \cdot f_S + dst \cdot f_D$

where:

$BSF = 0, BSI = 0 \Rightarrow f_S = 1$

$BSF = 1, BSI = 0 \Rightarrow f_S = \alpha$

$BSF = 0, BSI = 1 \Rightarrow f_S = 0$

$BSF = 1, BSI = 1 \Rightarrow f_S = 1 - \alpha$

$BDF = 0, BDI = 0 \Rightarrow f_D = 1$

$BDF = 1, BDI = 0 \Rightarrow f_D = \alpha$

$BDF = 0, BDI = 1 \Rightarrow f_D = 0$

$BDF = 1, BDI = 1 \Rightarrow f_D = 1 - \alpha$

Table 63.9 lists all possible color channel blend modes.

Table 63.9 Color channel blend modes

Mode	BSF	BSI	BDF	BDI	Blend equation
SRC_ONE DST_ONE	0	0	0	0	SRC + DST
SRC_ONE	0	0	0	1	SRC
SRC_ONE DST_ALPHA	0	0	1	0	SRC + DST × ALPHA
SRC_ONE DST_ONE_MINUS_ALPHA	0	0	1	1	SRC + DST × (1 - ALPHA)
SRC_ZERO DST_ONE	0	1	0	0	DST
SRC_ZERO DST_ZERO	0	1	0	1	0
SRC_ZERO DST_ALPHA	0	1	1	0	DST × ALPHA
SRC_ZERO DST_ONE_MINUS_ALPHA	0	1	1	1	DST × (1 - ALPHA)
SRC_ALPHA DST_ONE	1	0	0	0	SRC × ALPHA + DST
SRC_ALPHA	1	0	0	1	SRC × ALPHA
SRC_ALPHA DST_ALPHA	1	0	1	0	SRC × ALPHA + DST × ALPHA
SRC_ALPHA DST_ONE_MINUS_ALPHA	1	0	1	1	SRC × ALPHA + DST × (1 - ALPHA)
SRC_ONE_MINUS_ALPHA DST_ONE	1	1	0	0	SRC × (1 - ALPHA) + DST
SRC_ONE_MINUS_ALPHA	1	1	0	1	SRC × (1 - ALPHA)
SRC_ONE_MINUS_ALPHA DST_ALPHA	1	1	1	0	SRC × (1 - ALPHA) + DST × ALPHA
SRC_ONE_MINUS_ALPHA DST_ONE_MINUS_ALPHA	1	1	1	1	SRC × (1 - ALPHA) + DST × (1 - ALPHA)

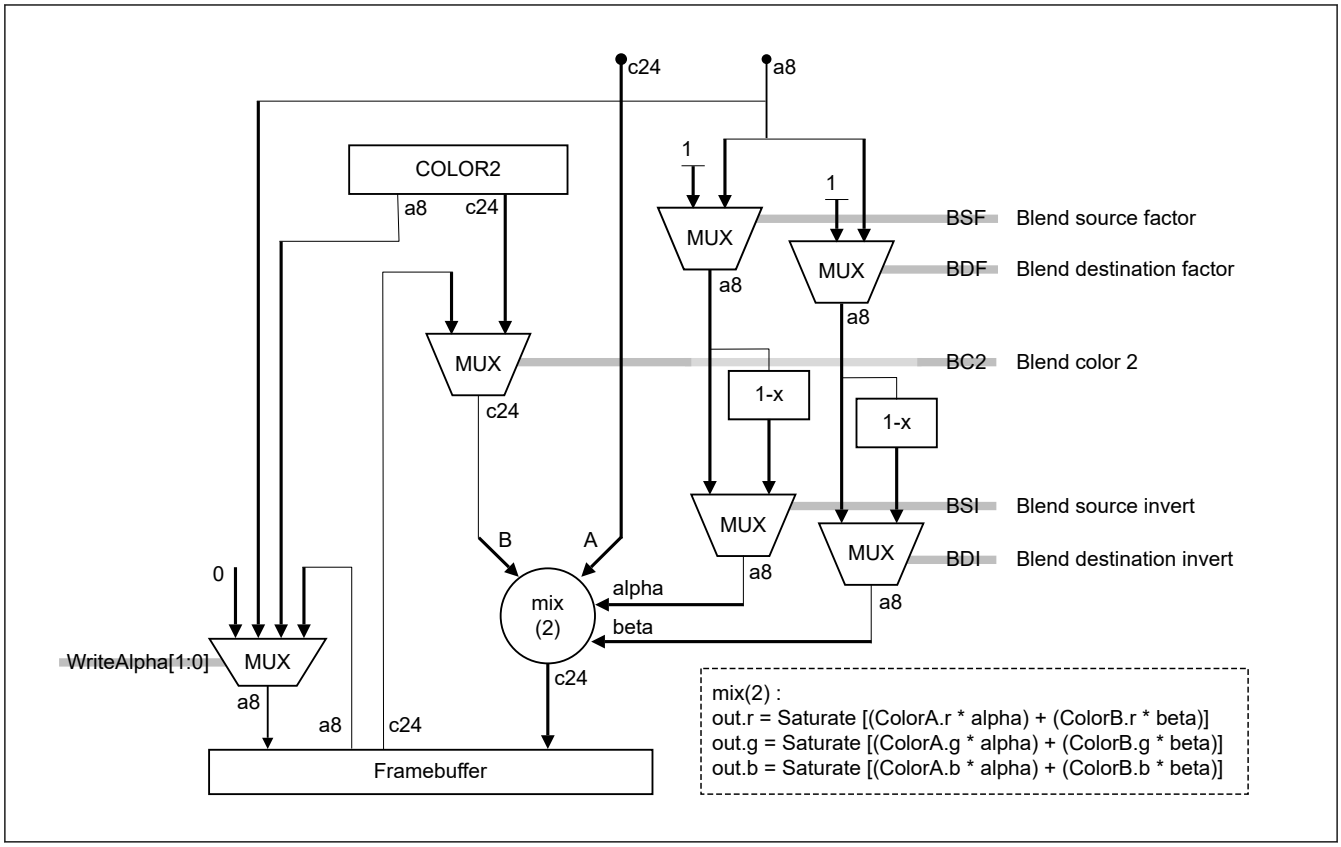


Figure 63.23 Color channel blend unit when CONTROL2.USEACB = 0

63.6.5.2 Alpha Channel Blending

The alpha channel can be blended in addition to the color channels. Alpha channel blending is enabled by setting CONTROL2.USEACB = 1. Alpha channel blending uses the same formulas and same blend modes as for the color channels. The alpha channel formulas can be set independently from the color channels.

The following alpha channel blend modes are supported:

- SRC_A_ZERO
- SRC_A_ONE
- SRC_A_SRC_A
- SRC_A_ONE_MINUS_SRC_A
- DST_A_ZERO
- DST_A_ONE
- DST_A_SRC_A
- DST_A_ONE_MINUS_SRC_A.

The alpha channel blend modes selected with the following flags:

- BSFA: blend source factor is SRC_A
- BSIA: blend source factor invert
- BDFA: blend destination factor is SRC_A
- BDIA: blend destination factor invert.

The formula for the blending is:

$dst\_alpha = src\_a \cdot f_{S\_a} + dst\_a \cdot f_{D\_a}$

where:

BSFA = 0, BSIA = 0 ⇒ $f_{S\_a} = 1$

BSFA = 1, BSIA = 0 ⇒ $f_{S\_a} = src\_a$

BSFA = 0, BSIA = 1 ⇒ $f_{S\_a} = 0$

BSFA = 1, BSIA = 1 ⇒ $f_{S\_a} = 1 - src\_a$

BDFA = 0, BDIA = 0 ⇒ $f_{D\_a} = 1$

BDFA = 1, BDIA = 0 ⇒ $f_{D\_a} = src\_a$

BDFA = 0, BDIA = 1 ⇒ $f_{D\_a} = 0$

BDFA = 1, BDIA = 1 ⇒ $f_{D\_a} = 1 - src\_a$

Table 63.10 lists all possible alpha channel blend modes.

Table 63.10 Alpha channel blend modes

Mode	BSF	BSI	BDF	BDI	Blend equation
SRC_A_ONE DST_A_ONE	0	0	0	0	SRC_A + DST_A
SRC_A_ONE	0	0	0	1	SRC_A
SRC_A_ONE DST_A_SRC_A	0	0	1	0	SRC_A + DST_A × SRC_A
SRC_A_ONE DST_A_ONE_MINUS_SRC_A	0	0	1	1	SRC_A + DST_A × (1 - SRC_A)
SRC_A_ZERO DST_A_ONE	0	1	0	0	DST_A
SRC_A_ZERO DST_A_ZERO	0	1	0	1	0
SRC_A_ZERO DST_A_SRC_A	0	1	1	0	DST_A × SRC_A
SRC_A_ZERO DST_A_ONE_MINUS_SRC_A	0	1	1	1	DST_A × (1 - SRC_A)
SRC_A_SRC_A DST_A_ONE	1	0	0	0	SRC_A × SRC_A + DST_A
SRC_A_SRC_A	1	0	0	1	SRC_A × SRC_A
SRC_A_SRC_A DST_A_SRC_A	1	0	1	0	SRC_A × SRC_A + DST_A × SRC_A
SRC_A_SRC_A DST_A_ONE_MINUS_SRC_A	1	0	1	1	SRC_A × SRC_A + DST_A × (1 - SRC_A)
SRC_A_ONE_MINUS_SRC_A DST_A_ONE	1	1	0	0	SRC_A × (1 - SRC_A) + DST_A
SRC_A_ONE_MINUS_SRC_A	1	1	0	1	SRC_A × (1 - SRC_A)
SRC_A_ONE_MINUS_SRC_A DST_A_SRC_A	1	1	1	0	SRC_A × (1 - SRC_A) + DST_A × SRC_A
SRC_A_ONE_MINUS_SRC_A DST_A_ONE_MINUS_SRC_A	1	1	1	1	SRC_A × (1 - SRC_A) + DST_A × (1 - SRC_A)

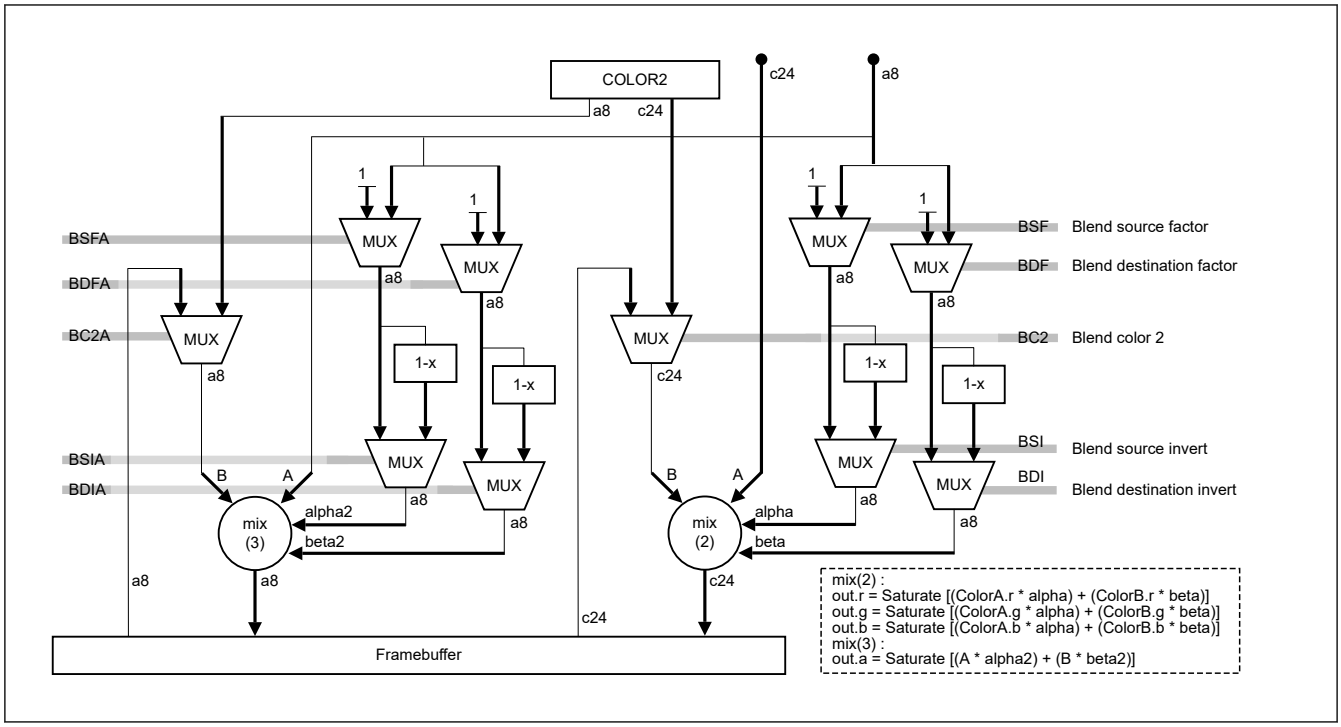


Figure 63.24 Alpha- and color-channel blend unit when CONTROL2.USEACB = 1

63.7 Rendering Modes

The rendering process can be performed in two different modes, register mode and display list mode.

63.7.1 Register Mode

In register mode, when operation is based on register settings, the host CPU configures and initiates each render process separately. To start a new render process, the host CPU must wait until the previous one is completed. In this mode, the host CPU is heavily engaged throughout the entire drawing procedure and is consequently to a large extent unavailable for other tasks.

The host CPU must set up all registers for performing a certain drawing operation before it can start the rendering process. A new register setup can only be started when the previous render process has completed. Before starting a new register setup, make sure that:

- STATUS.DLISTACTIVE = 0: Display list reader is idle
- STATUS.BUSENUM = 0: Pixel selection unit is idle.

Lastly, write the framebuffer start address to the ORIGIN register. This write triggers the 2D Drawing Engine to start rendering.

63.7.2 Display List Mode

In display list mode, the host CPU creates a display list in memory prior to starting the 2D Drawing Engine. Such a display contains a bundle of render operations. When started, the 2D Drawing Engine executes the display list autonomously in parallel with the host CPU, which is not involved with drawing operations most of the time. Use of a display list allows a fully asynchronous operation of the host CPU and the 2D Drawing Engine and offers the best possible system performance.

In this mode, the display list reader reads a memory block containing instructions on how to set the 2D Drawing Engine control registers and executes these control register writes accordingly.

(1) Display list start

To start execution of a display list, which already resides in memory, the start address of the display list is written to the display list start address register DLISTSTART. Because rewriting DLISTSTART also stops any ongoing display list execution, make sure that the previous display list process is completed by either of the following two methods:

- Check that STATUS.DLISTACTIVE = 0, which indicates idle status of the display list reader
- Wait for the display list interrupt DRWDLISTIRQ, which indicates completion of the previous display list process.

Caution: Direct writing to 2D Drawing Engine registers while the display list mode is active (STATUS.DLISTACTIVE = 1) might lead to a 2D Drawing Engine hang-up. To prevent this, always check that the display list reader is idle (STATUS.DLISTACTIVE = 0) before writing to any 2D Drawing Engine register.

(2) Display list format

Display lists are stored using direct register-to-value mappings, which means that the display list contains a one byte index that addresses a certain register and the value to be written to the register. The register index is derived from the address offset of the register address and can be calculated by dividing the address offset by 4. For the index of each register, see section 63.2. Register Descriptions.

As the 2D Drawing Engine registers are always 32 bits wide, each data unit (called a data word) to be written to a register is of the same size. An address word that contains the indices of the register to be written is stored in a packed notation with up to four indices stored in one 32-bit address word.

A display list command always starts with an address word, followed by up to four data words, one for each register. The indices are read and interpreted from LSB to MSB, so the register of the low byte index is written first.

(3) Example

In the following example:

- DWORD 0x201A_1930 // start of list address word
- DWORD 0x0000_0013 // data word 1 (for register 0x30)
- DWORD 0xFFFF_FFAA // data word 2 (for register 0x19)
- DWORD 0x4033_6480 // data word 3 (for register 0x1A)
- DWORD 0x0001_0000 // data word 4 (for register 0x20)
- DWORD... // next address word.

This stream of dwords updates the DRW registers as follows:

- Write 0x0000_0013 to register 0x30 = 48, which is IRQCTL
- Write 0xFFFF_FFAA to register 0x19 = 25, which is COLOR1
- Write 0x4033_6480 to register 0x1A = 26, which is COLOR2
- Write 0x0001_0000 to register 0x20 = 32, which is ORIGIN.

(4) Address word indices

Besides referencing a register, the indices of an address word can also have other meanings, depending on their value.

Table 63.11 Address indices function (1 of 2)

Index	Function	
0x00 to 0x7F	Register indices Two register indices trigger additional actions:	
	0x20 = 32:	A write to ORIGIN to set a new frame buffer address is delayed until the ongoing frame buffer write-back is complete, when STATUS.BUSYWRITE = 0.
	0x32 = 50:	A write to DLISTSTART sets a new display list start address stops the current display list and starts the new one.
0x80	Gap index, which is used to fill unused bytes of an address word. For example, if fewer than four indices are required, the remaining bytes are filled with 0x80. In this case, the number of the subsequent data words must be reduced accordingly.	

Table 63.11 Address indices function (2 of 2)

Index	Function	
0xFF	If the first index of an address word contains the special index 0xFF, the subsequent (second) index is interpreted as follows:	
	Bit [0] set:	Display list end.
	Bit [1] set:	Issue a full pipeline flush and wait (necessary before flip).
	Bit [2] set:	Wait for writeback complete (necessary before framebuffer format change).
	Bits [3:7]	Set all to 0s.
	Bit [1] and [2] settings are mutually exclusive. All indices after the special index 0xFF are ignored, and the next address word is read, if no display list end (bit [0] = 1) was set.	
	The remaining two indices must be set to 0x00.	

Caution : Gap indices 0x80 must not be placed between other indices, for example as “index1 - 0x80 - index3 - index4”. Always fill all indices after 0x80 with the gap index.

Caution : If any of the special indices 0x80 and 0xFF are used, no register index can follow after them in the same address word.

Table 63.12 2D Drawing Engine registers overview (1 of 2)

Register function	Symbol	Index
Control registers:		
Geometry control 0	CONTROL	0
Surface control	CONTROL2	1
Interrupt control	IRQCTL	48
Cache control	CACHECTL	49
Status control	STATUS	N/A*1
Hardware version and feature set ID	HWREVISION	N/A*1
Color registers:		
Base color	COLOR1	25
Secondary color	COLOR2	26
Pattern	PATTERN	29
Limiter registers:		
Limiter 1 start value	L1START	4
Limiter 2 start value	L2START	5
Limiter 3 start value	L3START	6
	L4START	7
Limiter 5 start value	L5START	8
Limiter 6 start value	L6START	9
Limiter 1 x-axis increment	L1XADD	10
Limiter 2 x-axis increment	L2XADD	11
Limiter 3 x-axis increment	L3XADD	12
Limiter 4 x-axis increment	L4XADD	13
Limiter 5 x-axis increment	L5XADD	14
Limiter 6 x-axis increment	L6XADD	15
Limiter 1 y-axis increment	L1YADD	16
Limiter 2 y-axis increment	L2YADD	17
Limiter 3 y-axis increment	L3YADD	18

Table 63.12 2D Drawing Engine registers overview (2 of 2)

Register function	Symbol	Index
Limiter 4 y-axis increment	L4YADD	19
Limiter 5 y-axis increment	L5YADD	20
Limiter 6 y-axis increment	L6YADD	21
Limiter 1 band width parameter	L1BAND	22
Limiter 2 band width parameter	L2BAND	23
Texture registers:		
Texture base address	TEXORIGIN	47
Texels per texture line	TEXPITCH	45
Texture size or texture address mask	TEXMASK	46
U limiter start value	LUSTART	36
U limiter x-axis increment	LUXADD	37
U limiter y-axis increment	LUYADD	38
V limiter start value integer part	LVSTARTI	39
V limiter start value fractional part	LVSTARTF	40
V limiter x-axis increment integer part	LVXADDI	41
V limiter y-axis increment integer part	LVYADDI	42
V limiter increment fractional parts	LVYXADDF	43
Color lookup table start address	TEXCLADDR	55
Write Data to DRWTEXCLADDR; after each data write, DRWTEXCLADDR is incremented by 1.	TEXCLDATA	56
Offset to the index for the indexed texture formats i8, i4, i2, and i1	TEXCLOFFSET	57
Compare value for R, G, B components of internal texel color representation.	COLKEY	58
Miscellaneous registers:		
Bounding box dimension	SIZE	30
Framebuffer pitch and spanstore delay	PITCH	31
Address of the first pixel in framebuffer	ORIGIN	32
Display list start address	DLISTSTART	50
Performance counters control	PERFTRIGGER	53
Performance counter 1	PERFCOUNT1	51
Performance counter 2	PERFCOUNT2	52

Note 1. These registers are read-only and cannot be accessed in display list mode, and they therefore have no index.

63.7.3 Stopping the Render Process

Stopping an ongoing render process requires a specific procedure, which is described in section 63.10. Stopping the 2D Drawing Engine Render Process.

63.8 Interrupts

The 2D Drawing Engine generates three interrupts:

- DRWBUSIRQ
- DRWENUMIRQ
- DRWDLISTIRQ.

63.8.1 Interrupt Sources

(1) DRWBUSIRQ

This is the 2D Drawing Engine bus error interrupt. It occurs when the 2D Drawing Engine attempts to access an undefined address range through the following registers:

- Framebuffer Base Address Register ORIGIN
- Texture Base Address Register TEXORIGIN
- Display List Start Address Register (DLISTSTART).

The bus error interrupt DRWBUSIRQ is then generated. The bus error interrupt only serves for debugging purposes. The interrupt source can be determined by the BUSERRMFB, BUSERRMTXMRL, and BUSERRMDL bits of the STATUS register.

Note: After a DRWBUSIRQ occurrence, you must apply a system reset.

(2) DRWENUMIRQ

This is the current render process finished interrupt.

(3) DRWDLISTIRQ

This is the display list interrupt. It is asserted on completion of a display list process. DRWDLISTIRQ is activated if one of the following conditions is true:

- The entire display list is complete
- The display list processing stops because a new display list start is triggered by a write to the Display List Start Address Register (DLISTSTART).

63.8.2 Interrupt Control

The three 2D Drawing Engine interrupts are combined into a single shared interrupt, DRW_IRQ, to the CPU. Each interrupt can be masked (disabled), or unmasked (enabled) by setting its associated enable bit in the Interrupt Control Register (IRQCTL).

The occurrence of an enabled interrupt (one with its mask bit set to 1 in IRQCTL) is monitored in the Status Control Register (STATUS) with its associated interrupt status bit is set to 1. The shared 2D Drawing Engine interrupt DRW_IRQ is then generated.

To clear the interrupt, the host CPU must write 1 to the interrupt clear bit in IRQCTL. The interrupt clear bit returns to 0 automatically.

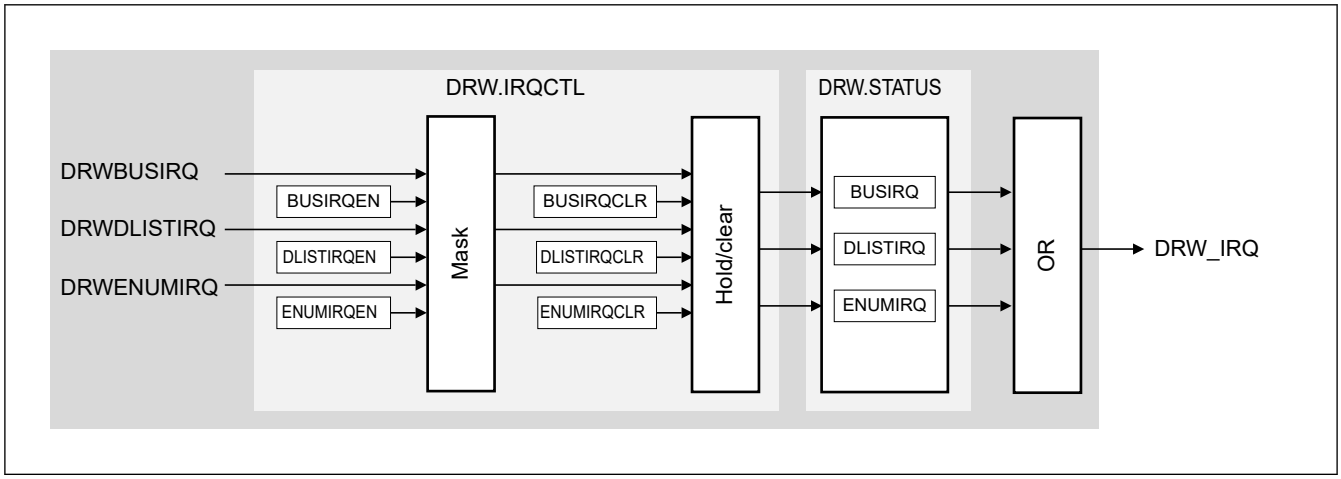


Figure 63.25 Interrupt Controller Unit (ICU)

63.9 Performance Counters

The 2D Drawing Engine features two independent 32-bit performance counter registers (PERFCOUNTk (k = 1, 2)) to count the number of occurrences of a certain event. The events to be counted can be set up independently for each performance counter register with the Performance Counter Control Registers, PERFTRIGGER.PERFTRIGGER2 for PERFCOUNT2 and PERFTRIGGER.PERFTRIGGER1 for PERFCOUNT1.

Table 63.13 lists the performance counter trigger events that can be selected.

Table 63.13 Performance counter trigger events

PERFTRIGGER.PERFTRIGGERk	Event
0	Disable performance counter
1	2D Drawing Engine active cycles
2	Framebuffer read access
3	Framebuffer write access
4	Texture read access
5	Invisible pixels (enumerated but selected with alpha 0%)
6	Invisible pixels while internal FIFO is empty (lost cycles)
7	Display list reader active cycles
8	Framebuffer read hits
9	Framebuffer read misses
10	Framebuffer write hits
11	Framebuffer write misses
12	Texture read hits
13	Texture read misses
31	Every clock cycle (for use as timer)

63.10 Stopping the 2D Drawing Engine Render Process

If a render process has started either in register or display list mode, the 2D Drawing Engine processes the data autonomously until the render process is finished. Depending on the rendering, this process might take several milliseconds.

If the 2D Drawing Engine is to be disabled because, for example, the MCU enters a low power mode, proceed as follows to stop the ongoing rendering:

1. Set the following registers as follows:
   SIZE = 0x00010001
   Set bounding box dimensions to 1 pixel x 1 line.
   CONTROL2 = 0x00000000
   Color format A (8), no texture, CLUT.
   ORIGIN = UnmappedAddress
   The UnmappedAddress is an address that is not available for 2D Drawing Engine access.
   The recommended UnmappedAddress is given here under the key word “UnmappedAddress”.
   Alternatively do the same in display list mode:
   DWORD 0x8020011E // start of “address word” list
   DWORD 0x00010001 // SIZE = 0x00010001
   DWORD 0x00000000 // CONTROL2 = 0x00000000
   DWORD UnmappedAddress // ORIGIN = UnmappedAddress
2. Wait for the Bus Error corresponding to the unmapped address violation, which indicates access to an unmapped address and the stop of the render process.
   UnmappedAddress = 0xFFFFFFF0
3. Disable the 2D Drawing Engine, if wanted.

63.11 MathML Sample

#1

$y = \widetilde{a}x + \widetilde{b}$

#2

$0 = f(x,y) = \widetilde{a}x - y + \widetilde{b} = \mathrm{ax} + \mathrm{by} + c$

with $a = \widetilde{a}$, $b = -1$, $c = \widetilde{b}$

#3

$\overrightarrow{p} = \begin{pmatrix} x \\ y \end{pmatrix}, \overrightarrow{n} = \begin{pmatrix} a \\ b \end{pmatrix} \Rightarrow f(x,y) = \mathrm{ax} + \mathrm{by} + c = \overrightarrow{p} \cdot \overrightarrow{n} + c$

#4

The projection of $\overrightarrow{p} - \overrightarrow{p}_0$ to $\overrightarrow{n}$ is the distance d of the point P to the line. In this case, f (x, y) describes the distance to the line of the pixel with coordinates (x, y).

#5

$\Delta\overrightarrow{p} = \overrightarrow{p}_1 - \overrightarrow{p}_0 = \begin{pmatrix} x_1 - x_0 \\ y_1 - y_0 \end{pmatrix} = \begin{pmatrix} \Delta\mathrm{x} \\ \Delta\mathrm{y} \end{pmatrix}$

#6

Consider the equation for a circle with the center at $\overrightarrow{c} = \begin{pmatrix} s \\ t \end{pmatrix}$ and radius r:

#7

$0 = f(x,y) = (x-s)^2 + (y-t)^2 - r^2$

#8

$0 = f(x,y) = \overrightarrow{p}_0 \cdot \overrightarrow{n} + c \Rightarrow c = -\overrightarrow{p}_0 \cdot \overrightarrow{n}$

#9 OLD

$\overrightarrow{p}_0 = \begin{pmatrix} x_0 \\ y_0 \end{pmatrix} \Rightarrow \widetilde{(\overrightarrow{p}_0)} = \begin{pmatrix} u_0 \\ v_0 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$

#9 NEW

$\overrightarrow{p}_0 = \begin{pmatrix} x_0 \\ y_0 \end{pmatrix} \Rightarrow \left(\widetilde{\overrightarrow{p}_0}\right) = \begin{pmatrix} u_0 \\ v_0 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$

#10 OLD

$\overrightarrow{n} = \begin{pmatrix} -\Delta\mathrm{y} \\ \Delta\mathrm{x} \end{pmatrix} / \left(\sqrt{\Delta x^2 + \Delta y^2}\right)$

#10 NEW

$\overrightarrow{n} = \begin{pmatrix} -\Delta\mathrm{y} \\ \Delta\mathrm{x} \end{pmatrix} \cdot \frac{1}{\sqrt{\Delta x^2 + \Delta y^2}}$

#11

$\begin{bmatrix} a & b \\ c & d \end{bmatrix}$

#11-2

$\begin{bmatrix} m_{11} & m_{12} \\ m_{21} & m_{22} \end{bmatrix}$

#12

$\mathrm{start2} = \mathrm{xadd1}$

#13

$f(x+1, y)$

#14

$$\begin{pmatrix} \mathrm{w} \\ 0 \end{pmatrix} = A \cdot \begin{pmatrix} m_{11} \\ m_{12} \end{pmatrix} \wedge \begin{pmatrix} 0 \\ h \end{pmatrix} = A \cdot \begin{pmatrix} m_{21} \\ m_{22} \end{pmatrix}$$

#15

$$\mathrm{sqrt}\left(x^2 + y^2\right) = \frac{\sqrt{x^2 + y^2}}{k}$$

#16

$$k = \prod_{i=0}^{\infty} \frac{1}{\sqrt{1 + 2^{-2i}}} \fallingdotseq 0.607252935008881256169 4$$

63.12 Usage Notes

63.12.1 Power Gating Control or Software Standby Mode

Before transitioning to graphics power domain off or software standby mode, stop the operation as described in the following sections.

See section 63.10. Stopping the 2D Drawing Engine Render Process.

After returning from graphics power domain off or software standby mode, resume operation as described in the following sections.

See section 63.7. Rendering Modes.

63.12.2 Module-Stop Function Setting

DRW operation can be disabled or enabled using Module Stop Control Register C (MSTPCRC). The DRW module is initially stopped after reset. Releasing the module-stop state enables access to the registers.